References

Reference	Author
Spring Batch Introduction	Spring
Spring Batch Domain Language	Spring
Spring Batch Architecture	Spring
Cron Expressions	Spring
Batch Processing Gazette Article	David Jobrant
DD130 - Multitenancy	Netcompany
DD130 - Batch Job Reporting	Netcompany

Introduction

Target Audience

The target audience for this documentation is project architects and developers who should gain knowledge about batch processing in Foundation before integrating it to suit their needs in their projects. It is also relevant to developers who are continuing the development of the batch engine, or developers who want to gain knowledge to be able to contribute to it.

Purpose

This document aims to provide an overview of the batch engine and guide you to relevant information. The batch interfaces and classes in the code provide thorough Javadoc documentation, so please also refer to those.

Developer Requirements

- An abstract understanding of batch processing. What is it and why do we use it? A good place to start, which also introduces concepts of Spring Batch and Foundation Batch, is the Gazette article "Batch Processing - the Organisation's Night Shift".
- To understand how Foundation Batch works, it's important to first grasp the core concepts and architecture of Spring Batch that we've extended and utilized. Here are some good resources for learning about it:
  - Spring Batch Introduction for background about why and when batch and bulk processing is useful.
  - Spring Batch Architecture for an overview of the architecture.
  - Spring Batch Domain Language for learning the terminology, which is essentially the same in Foundation Batch. We will use this terminology throughout this document.
- A basic understanding of cron expressions: Cron Expressions.

Background Information

Batch processing refers to the automated, non-interactive execution of a set of tasks, often performed on large volumes of data. It is commonly used for tasks that are repetitive, resource-intensive, or time-consuming, such as report generation, nightly calculations, or integration syncs. These jobs typically run on a schedule or are triggered by specific events, and they often run without direct user interaction. The primary benefits of batch processing include:

- **Automation**: Reduces the need for manual intervention in recurring tasks.
- **Efficiency**: Optimizes resource usage by executing tasks in bulk.
- **Scalability**: Handles large datasets with predictable performance.
- **Reliability**: Ensures consistent and repeatable outcomes through error handling and job tracking.

To standardize and simplify batch development across projects, the Foundation platform builds on top of Spring Batch. Foundation Batch integrates Spring Batch into our ecosystem and introduces conventions, utilities, and configuration layers for creating, scheduling, and running batch jobs.

This document provides a technical overview of our batch engine implementation and is intended to help projects understand how to use and customize various aspects of batch processing and create batch jobs using Foundation Batch. For deeper insight into the specific components and behavior, please also refer to the relevant codebase and Javadoc documentation.

Overview of Spring Batch

Spring Batch is a robust and extensible framework for building batch processing applications in Java. It provides a set of prebuilt components and infrastructure abstractions designed to simplify the development of high-performance, reliable, and maintainable batch jobs.

At its core, Spring Batch handles the orchestration of batch workflows by managing job execution, transaction boundaries, state persistence, retry and skip logic, and job metadata. It also includes built-in support for common data processing patterns such as reading from flat files, transforming records, and writing to databases or other systems.

Spring Batch is particularly well-suited for applications that:

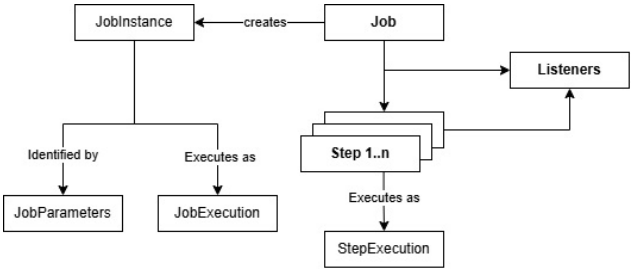
- Process large datasets in a controlled and repeatable manner
- Require job state tracking and restart capabilities
- Benefit from modular, step-based job decomposition
- Need transaction and resource management out-of-the-box

Spring Batch Terminology

Component	Description
Job	A container for one or more steps, representing an end-to-end batch process
JobInstance	A specific invocation of a job, uniquely identified by JobParameters
JobParameters	Immutable key-value pairs that define or distinguish job executions
JobExecution	A single attempt to run a JobInstance, with execution status and metrics

Step	A discrete unit of work within a Job
StepExecution	Execution context for a Step, including tracking metrics and status
Reader	Component responsible for reading input data
Processor	(Optional) Component for transforming input data
Writer	Component responsible for writing the output
Listeners	Interceptors for hooking into job or step lifecycle events
Aggregator	Aggregate the result of partition worker step to the result of the partition manager step

The diagram below shows the relationship between these components:



A batch job and its related components.

For a more detailed explanation of terminology, see the official Spring Batch documentation on Domain Language.

High-Level Description of the Component

The batch library in Foundation provides a simple and well-structured solution for projects to use when implementing batch jobs with their own business logic, such as sending/receiving data to external parties or calculating benefits for a large number of citizens. The batch engine is ideal for purposes that should handle large amounts of processing on a scheduled basis. The functional purpose of the batch component is to provide systems with a loosely coupled framework for scheduling and executing processes manually or automatically within specified schedules.

The batch library in Foundation is split into four modules: Core, Engine, Item, and Scheduler. These modules provide functionality such as error handling, scheduling, management, and creation of batch jobs and steps.

Alongside the batch engine, an administration user interface is provided to let users handle configuration, scheduling, execution, and overall management and status monitoring of the batch jobs that are running on their system.

Foundation Batch Job Structure

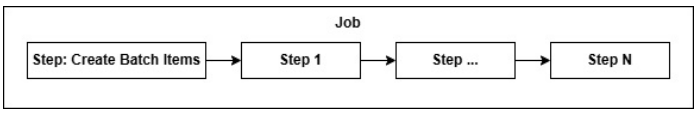
The following sections detail the structure of a batch job in Foundation.

Structure of a Job

Each Foundation batch job consists of a series of sequential steps, meaning that each task is processed in a set order where one step has to finish before the next can begin. The job begins with a **Create Batch Items** step, which generates the initial set of **batch items**, the units of work that subsequent steps will process.

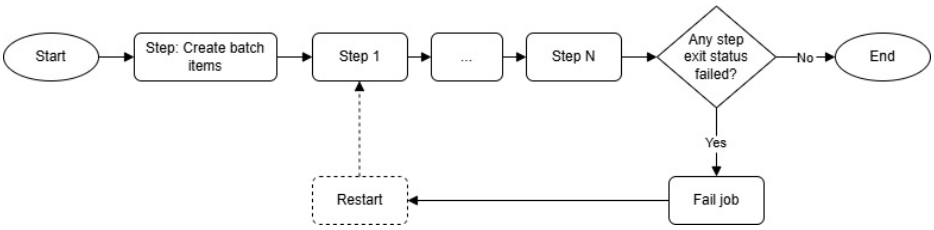
> **What is a batch item?** In Foundation, a batch item represents an individual unit of work in your batch processing system. It serves as a wrapper around the actual data being processed, while also maintaining metadata about the processing status and context. A batch item exists within the context of a specific step in a job instance. Learn more about batch items in section [BatchItem](#batchitem).

After the initial step, a job consists of an additional **arbitrary number of steps**. These steps will only act on previously created batch items but can create new items for usage in future steps.



Structure of a batch job.

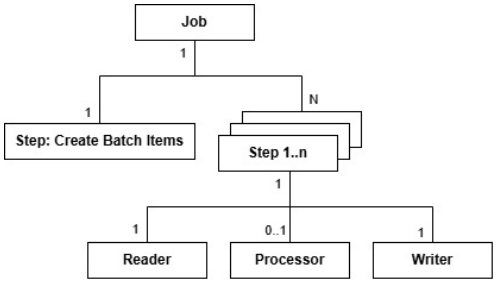
The flow diagram below illustrates how execution progresses through the job. The create batch items step is executed only once per job instance to ensure idempotency and is not re-run on job restarts. If processing fails for one or more items, the job continues processing the rest but will be marked as failed until all items reach a terminal state. On job restart, all steps except the first are re-executed. Steps that have already completed processing are skipped unless they contain unfinished items. For more information about error handling, refer to the section Error Handling.



Flow of a batch job from start to end, with error handling.

Structure of a Step

A **step** is a distinct phase within a batch job that encapsulates a specific task in a defined sequence. It contains a **reader**, an **optional processor**, and a **writer** component. A step will only process batch items that are either new or have failed in a previous execution.



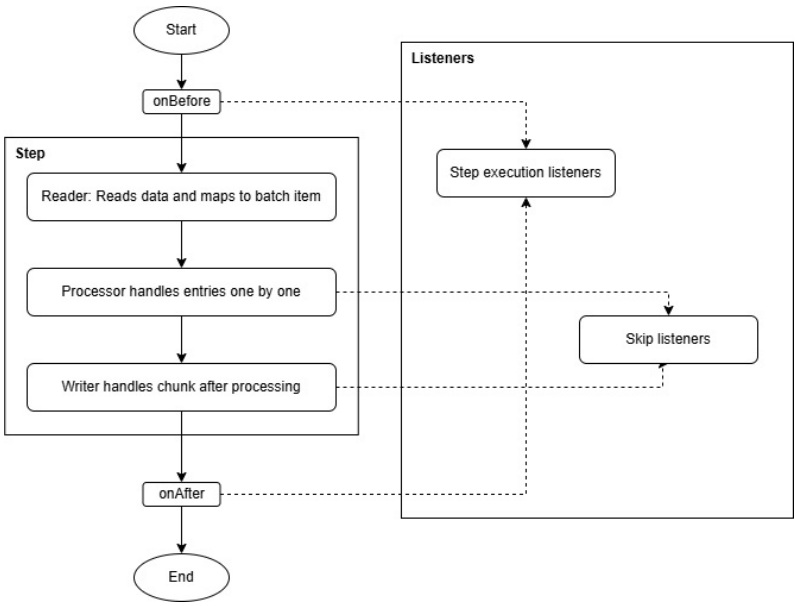
How a job relates to its steps and step components.

Step Components

The components of a step are introduced below. For guidance on implementing them, refer to the Item package.

- **Reader**: The reader component acts as the entry point for data flowing through the step and is responsible for retrieving batch items that will be processed in your batch job. It reads a chunk of N entries at a time.
- **Processor**: The processor contains your core business logic that transforms input data into output. The processor will handle one entry from a chunk at a time, and order is not guaranteed. Processors should be **idempotent**, which means that running the processing just once or ten times should yield the same result. This ensures it is safe to have multiple attempts in case something goes wrong. Therefore, ideally, there should be no side effects.
- **Writer**: The writer is responsible for creating new batch items for future steps and performing side effects. Often, that means persisting the data somewhere. It is the developer's responsibility to ensure it is idempotent if you go outside the transaction of the job.
- **Listeners**: Listeners allow for reuse of code and are used to intercept execution of jobs and steps. Steps deal with failures through a listener. For example, if a batch skippable exception is thrown, the item will be marked as failed and can later be restarted. For more information about the listener logic, see the Error handling section.
- **Aggregators:** Aggregators is used in partitioned step to aggregate the metadata (status, exit status, ...) and data (for aggregation step) of the partition worker to the final result of the partition manager.

The diagram below shows the flow of the step and how the listeners are connected to the step.



Flow of a batch step.

Step Independence

Each step in a batch job should be designed to operate independently of other steps, beyond the creation of new batch items. This independence is critical because:

- It prevents cascading failures between steps
- It enables clean restart capabilities if a job fails
- It simplifies debugging and troubleshooting

When steps are interdependent, failures become more complex to handle. For example, if the first step succeeds but the second fails, rerunning the job might cause the first step to process data differently than in the original run.

Step Partitioning

It is possible to partition a step, see section partitioning. This is useful if the batch job will handle a large amount of data or work. Even when partitioned, the step's logical structure and behavior remain unchanged; processing simply happens concurrently.

Batch Job Configuration Files

A batch job in Foundation is created in a batch job configuration file. Here, you define your job, steps and beans of components required for the execution of your batch job.

Technical Architecture

The batch package is split into seven main modules, each responsible for a certain aspect of the batch component. Each module is described below in their own sections.

- **Core**: Core functionality of batch. That includes service classes used in the other modules, infrastructure for JobParameters, generic classes for `BatchJob` and `BatchStep`, classes for configuring batch jobs and error-handling. For details, see section Core.
- **Engine**: Management of batch jobs and steps, such as starting, stopping, and restarting batch jobs. Also includes throttling services to control batch job acceptance based on available system resources. For details, see section Engine.
- **Item**:

- **Service**: Services for handling the batch item lifecycle, both for `BatchItems` and `AggregateItems`.
- **Components**: Builders for steps and jobs, readers, processors, and writers. For details, see section Item. This module contains most of the abstractions that you will use when implementing your own batch jobs.

- **Scheduler**: Provides classes for scheduling of batch jobs. For details, see section Scheduler.
  - **Service**: Services for retrieving, updating and managing schedules. Contains the queue which job commands are added to.
  - **Producer**: The scheduler, responsible for adding job command messages based on active schedules to the queue.
  - **Consumer**: The consumer, which polls job command messages from the queue and handles them.

Core

Link to core package

This package contains generic classes for batch jobs and batch steps, and error handling. It is also responsible for retrieving batch job configuration parameters from the system parameters. The `BatchJob` and `BatchStep` classes are used when defining batch jobs and steps in a configuration file for a batch job.

BatchCoreConfig

A Spring configuration class that sets up the core batch functionality from Spring. The purposes of the annotations are:

- @EnableIntegration - Enables Spring Integration messaging framework
- @EnableBatchIntegration - Bridges Spring Batch with Spring Integration, and provides beans for partitioning such as `RemotePartitioningWorkerStepBuilder`

Bean	Type	Purpose
`batchMessageStoreJdbcTemplate`	`JdbcTemplate`	Database operations for batch message storage with single row fetch
`jobParametersValidatorProvider`	`BatchJobAssociatedProvider<BatchJobParametersValidator>`	Provides job-specific parameter validators
`jobParametersProvider`	`BatchJobAssociatedProvider<JobParameters>`	Provides job-specific parameters
`jobParametersExtraServiceProvider`	`BatchJobAssociatedProvider<JobParametersExtraService>`	Provides additional job parameter services
`batchContextRunner`	`BatchContextRunner`	Executes batch jobs with proper context initialization
`contextProvidingJobExecutionListener`	`ContextInformationProviderJobExecutionListener`	Job-level listener for context information management
`contextProvidingListener`	`ContextProvidingStepExecutionListener`	Step-level listener for context provision (step-scoped)

Batch Core Services

For an explanation of the batch core services, please see the provided JavaDoc here or in the code.

BatchJobConfigurationService

```
/**
 * This service is responsible for fetching batch job configuration parameters from the system parameters.
 */
public interface BatchJobConfigurationService {

    /**
     * Fetches batch job configuration parameters from the system parameters.
     * @param parameterInstanceKey the system parameter instance key.
     * @return an optional containing the batch job configuration for the corresponding system parameter instance.
     */
    Optional<BatchJobConfiguration> getBatchJobConfigurationParameter(String parameterInstanceKey);
}
```

BatchJobRegistryService

This service wraps the `BatchJob` enum to provide controlled access to batch jobs.

```
/**
 * A service interface that wraps {@link BatchJob} to provide controlled access to batch jobs. This allows filtering or restricting the list of batch
jobs
 * that can be retrieved, for example, based on the current user, tenant, or other contextual information.
 * <p>
 * Typical implementations may enforce security, multi-tenancy, or other business rules to ensure
 * only authorized jobs are accessible.
 */
public interface BatchJobRegistryService {

    /**
     * Retrieves all batch jobs visible and accessible in the current context (e.g., for the current user or tenant).
     *
     * @return a {@link List} of {@link BatchJob} instances representing the accessible batch jobs;
     * never {@code null}, but may be empty if no jobs are available
     */
    List<BatchJob> getBatchJobs();

    /**
     * Retrieves a batch job by its name, if it is accessible in the current context.
     *
     * @param name the name of the batch job to retrieve (must not be {@code null})
     * @return an {@link Optional} containing the {@link BatchJob} if found and accessible,
     * or an empty {@link Optional} if not found or not accessible
     */
    Optional<BatchJob> getBatchJob(String name);
}
```

BatchJobOperatorService

Service that provides immediate, direct control over batch job execution, bypassing the scheduler queue to enable instant attempts of job termination through the Spring `JobOperator` without waiting for worker availability.

```
/**
 * Service interface that provides high-level operations for controlling Spring Batch jobs.
 * <p>
 * This abstraction wraps the Spring Batch {@link org.springframework.batch.core.launch.JobOperator}
 * to expose functionality for initiating a graceful stop of a running job instance.
 * </p>
 *
 * @see org.springframework.batch.core.launch.JobOperator
 */
public interface BatchJobOperatorService {

    /**
     * Initiates a graceful stop of the running job execution associated with the given job instance id.
     *
     * @param jobInstanceId the ID of the job instance whose execution should be stopped (must not be null)
     * @return {@code true} if a stop request was successfully issued, {@code false} otherwise
     * @throws NoSuchJobExecutionException     if no execution exists for the given job instance
     * @throws JobExecutionNotRunningException if the job execution exists but is not currently running
     */
    boolean initiateStopJob(@NotNull Long jobInstanceId)
            throws NoSuchJobExecutionException, JobExecutionNotRunningException;
}
```

BatchJobTenantService

Service for managing tenant access and visibility for batch jobs.

> **Note:** There is no default implementation of this service in Foundation, it needs to be provided by projects. An example implementation has been added to the Reference application. It is only needed to create an implementation in a multitenant system.

```
/**
 * Service for managing tenant access and visibility for batch jobs.
 * <p>
 * Provides multi-tenancy support by determining which tenants are allowed to access specific batch jobs,
 * and whether the current context (e.g., user or tenant) has access to a given batch job.
 */
public interface BatchJobTenantService {

    /**
     * Retrieves the list of tenant identifiers that have access to the specified batch job.
     *
     * @param batchJob the batch job for which to retrieve accessible tenants (must not be {@code null})
     * @return a {@link List} of tenant identifiers (never {@code null}, but may be empty)
     */
    List<String> getAccessibleTenants(@NotNull BatchJob batchJob);

    /**
     * Retrieves a map of tenant identifiers that have access to the specified batch job.
     * Key should be the id of the tenant, and the value should be a human-readable value
     *
     * @param batchJob the batch job for which to retrieve accessible tenant options (must not be {@code null})
     * @return a {@link Map} of tenant identifiers as keys and human-readable string as value (never {@code null}, but may be empty)
     */
    Map<String, String> getAccessibleTenantOptions(@NotNull BatchJob batchJob);

    /**
     * Checks if the current context (user/tenant) has access to the specified batch job.
     *
     * @param batchJob the batch job to check access for (must not be {@code null})
     * @return {@code true} if the current context has access; {@code false} otherwise
     */
    boolean isAccessibleByCurrentTenant(@NotNull BatchJob batchJob);
}
```

Error Handling

This package contains the class `BatchException` along with extension classes that are used for throwing `BatchErrorCodes` when certain exceptions occur.

BatchException

The `BatchException` classes are described in the table below.

Exception	Description
`BatchException`	Base exception class for batch operations
`BatchRetryableException`	For errors related to retry logic
`BatchSkippableException`	For errors related to skip logic
`BatchUnrecoverableException`	For fatal errors that cannot be recovered from

Error Codes

`BatchErrorCodes` contains batch-related error codes in the following format: `8XXX`. There are subcategories for batch errors. An exhaustive list of the batch subcategory-related errors can be found below.

Error Number Range	Group Description
8001-8099	Engine-related errors
8100-8199	Scheduling-related errors
8200-8299	Batch execution-related errors
8300-8399	Batch item-related errors
8400-8499	Batch job parameter-related errors

BatchJob and BatchStep

The `BatchJob` and `BatchStep` classes are used to internally represent a job and step respectively. `BatchJob` represents a batch job. Each job has a unique identifier and contains a sequence of `BatchStep`s. The job along with its steps are typically defined in a constants file for each unique batch job.

Abstract Job and Step Builders

The package contains `AbstractBatchJobBuilder` and `AbstractBatchStepBuilder`. Both provide general frameworks for how these job and step builders should be structured. The former provides an abstract builder for constructing batch job configurations. This builder class encapsulates common elements and behaviors necessary for setting up batch jobs abstraction. The latter provides a generic abstract builder for creating batch processing steps within a job.

JobParameters

For each batch job, you must implement a `{BatchJobName}JobParameters` class that extends `JobParameters` from Spring Framework. This class serves as a strongly-typed container for all parameters that your batch job requires during execution.

The job parameters class must additionally implement two key interfaces:

- **`OrderedJobParameters`**: This interface ensures consistent ordering of parameters in the dynamically generated parameter tables within the batch administration UI. The `getParameterOrder()` method returns a list of parameter names in the desired display order.
- **`OptionalJobParameters`**: This interface offers the ability to specify which job parameters that are not mandatory, ensuring they can be treated as optional during form generation and validation, using the method `getOptionalParameters()`.
- **`AssociationMethod`**: This interface provides the mechanism for identifying the relevant `JobParameters` bean from the job type name. The `isAssociated()` method establishes the connection between your batch job definition and its corresponding parameter class.

OrderedJobParameters interface

```
/**
 * A mixin interface that provides parameter ordering functionality for batch jobs
 * within the foundation framework.
 * <p>
 * This interface defines capabilities that can be mixed into job-specific parameter implementations,
 * offering parameter ordering functionality that defines the intended sequence of parameters.
 * <p>
 * Example usage:
 * <pre>
 * {@code
 * @Component
 * public class MyJobParameters implements OrderedJobParameters {
 *     @Override
 *     public List<String> getParameterOrder() {
 *         return List.of("param1", "param2");
 *     }
 * }
 * }
 * </pre>
 *
 * @see org.springframework.batch.core.JobParameters
 * @see nc.foundation.libraries.batch.core.service.model.types.BatchJob
 */
public interface OrderedJobParameters {

    /**
     * Returns an ordered list of parameter names that defines the sequence
     * in which parameters should be presented or processed.
     *
     * @return a list of parameter names in the desired order
     */
    List<String> getParameterOrder();
}
```

OptionalJobParameters interface

```
/**
 * A mixin interface that provides optional parameter functionality for batch jobs
 * within the Foundation for Java framework.
 * <p>
 * This interface defines capabilities that can be mixed into job-specific parameter implementations,
 * offering the ability to specify which parameters should be treated as optional during form generation
 * and validation. Parameters marked as optional will not be required for job execution and can be
 * left empty in the dynamic batch job forms.
 * <p>
 * Example usage:
 * <pre>
 * {@code
```

```
 * @Component
 * public class BatchItemDeletionBatchJobParameters extends JobParameters
 *     implements OptionalJobParameters, OrderedJobParameters, AssociationMethod {
 *
 *     @Override
 *     public List<String> getOptionalParameters() {
 *         return List.of("param3"); // param3 is optional
 *     }
 *
 *     @Override
 *     public List<String> getParameterOrder() {
 *         return List.of("param1", "param2", "param3");
 *     }
 * }
 *
 * @see OrderedJobParameters
 * @see nc.foundation.libraries.batch.core.service.api.AssociationMethod
 * @see org.springframework.batch.core.JobParameters
 * @see nc.foundation.libraries.batch.core.service.model.types.BatchJob
 */
public interface OptionalJobParameters {

    /**
     * Returns a list of parameter names that should be treated as optional
     * during form generation and validation.
     * <p>
     * @return a list of parameter names that are optional; never {@code null}
     * but may be empty if all parameters are mandatory
     */
    List<String> getOptionalParameters();
}
```

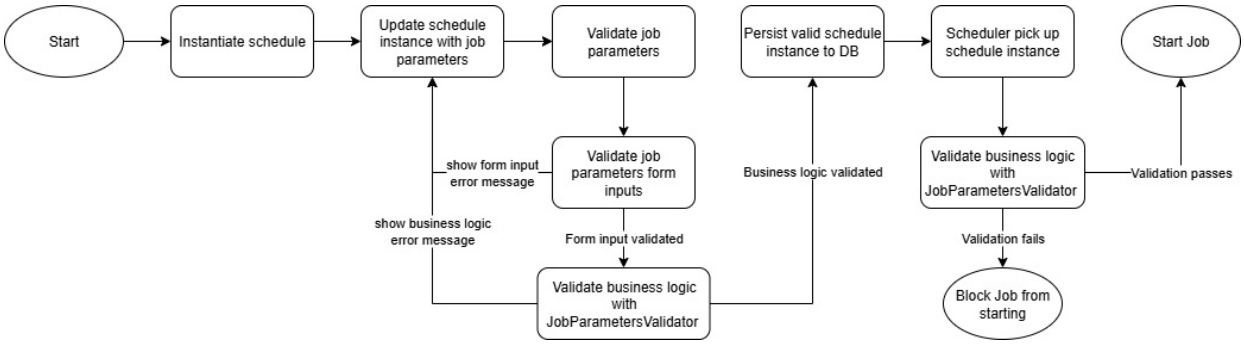
AssociationMethod interface

```
/**
 * Interface defining a method to determine associations between implementations and batch jobs.
 */
public interface AssociationMethod {
    /**
     * Determines whether this implementation is associated with the specified batch job.
     * <p>
     * This method enables the dynamic discovery of job-specific implementations
     * by services like {@code JobParametersFormProvider} that need to find the appropriate
     * parameter set for a given job.
     *
     * @param jobConstant the batch job to check association against, must not be null
     * @return {@code true} if this implementation is associated with the specified job,
     *         {@code false} otherwise
     * @throws NullPointerException if {@code jobConstant} is null
     */
    boolean isAssociated(@NotNull BatchJob jobConstant);
}
```

For batch jobs that do require parameters, you would populate the constructor with the appropriate parameter map and return the parameter names in `getParameterOrder()`.

Job Parameters Validation Flow

The following graph illustrates the JobParameters lifecycle.



It shows how JobParametersValidator is run both right before the call to `.startJob()` and as the form is being validated. This is done to ensure we only persist sound job parameters.

ParameterConverter

The `ParameterConverter` interface in Foundation Batch has a specific signature that differs from the standard Spring Batch approach. It takes existing `JobParameters` as input and returns enhanced `JobParameters`, allowing for parameter transformation, validation, and the addition of system-generated parameters.

The converter performs several important functions:

1. **Parameter Enhancement**: Adds system-generated parameters like timestamps or unique identifiers
2. **Parameter Transformation**: Converts or normalizes existing parameters
3. **Validation**: Ensures that provided parameters meet business rules and constraints
4. **Uniqueness Control**: Manages job instance uniqueness through identifying parameters

The converter must also implement `AssociationMethod` to link it to the correct batch job type.

The `convert` method receives the original `JobParameters` (which may contain user-provided parameters from the administration UI) and returns an enhanced version with additional system parameters. If you have optional job parameters, these can also be added or removed from the job parameters object based on if they are present or not. The usual parameter we enhance in the `convert` method is the dateTime parameter - which we use to ensure that no more than one job with the same parameters is run each second.

> **Note:** The identifying parameter (marked with `true` as the fourth argument) helps Spring Batch determine job instance uniqueness - jobs with the same identifying parameters are considered the same instance.

JobParametersValidator

A `JobParametersValidator` is used to validate the given JobParameters when trying to start a job. It ensures that the provided parameters are valid, complete, and meet the business requirements for the specific batch job.

Foundation Batch provides an abstract `BatchJobParametersValidator` class that includes common validation methods and implements the `AssociationMethod` interface to link validators to specific batch jobs. Every batch job should extend this base class to implement its own validation logic.

The base validator provides two utility methods:

- **`isJobParametersObjectNull()`** : Validates that the JobParameters object itself is not null
- **`areJobParametersMissing()`** : Checks that all required parameters are present in the JobParameters

The validator is automatically discovered and used by the batch framework during job startup. It runs before the `ParameterConverter` and ensures that only valid parameters proceed to job execution. The validator is configured on the job creation in the batch job configuration file through the job builder's `.validator()` method.

For batch jobs with optional parameters, the list of required keys has to be filtered based on the list of optional parameters from the `OptionalJobParameters` interface before calling `areJobParametersMissing()` . An example of this can be seen in the BatchItemDeletion batch job.

For batch jobs with no parameters (like the TaskFilter example batch job), the validation typically only checks for null parameters and ensures no unexpected parameters are provided, since `getParameterOrder()` returns an empty list.

> **Note:** On any `batchEngine.startJob()` execution of a `Job` that has a `{BatchJobName}JobParametersValidator` attached, the validation will be run just before execution.

Engine

[Link to engine package]

The engine is a thin wrapper around Spring Batch, responsible for management of batch jobs and steps. It contains configuration that customizes the auto-configured `jobLauncher` and defines a custom `TaskExecutor` for it. Additionally, the engine defines beans required for management of partitioned batch jobs.

BatchEngineConfig

A Spring configuration class that sets up key components of the batch engine architecture.

This configuration relies on the default `JobLauncher` provided by Spring Boot's `BatchAutoConfiguration`. To inject a custom thread pool, it defines a `BeanPostProcessor`. This processor intercepts the auto-configured `JobLauncher` after its creation and replaces its default task executor with the custom `batchEngineTaskExecutor`.

Its purpose is to:

- Customize the task executor of the auto-configured Spring Batch `JobLauncher`
- Define a dedicated thread pool for job execution
- Provide other core engine services like parameter conversion and job throttling

Bean	Type	Purpose
`jobLauncherTaskExecutorBeanPostProcessor`	`BeanPostProcessor`	Intercepts and customizes the `JobLauncher` with a specific `TaskExecutor`
`batchEngineTaskExecutor`	`ThreadPoolTaskExecutor`	A custom thread pool for job execution and remote partitioning
`parameterConverterProvider`	`BatchJobAssociatedProvider<ParameterConverter>`	Provides job-specific parameter converters to the engine
`batchEngineThrottleService`	`BatchEngineThrottleService`	Controls batch job acceptance based on system resource availability

BatchEngine

The main responsibility of the batch engine is to handle starting, restarting, stopping, and recovering stuck batch jobs.

> **Note:** Only one job execution can run for each job instance at a time.

For an explanation of the engine service, please see the provided JavaDoc here or in the code.

```
public interface BatchEngine {

    /**
     * Provides the ability to query the engine to see if it has available capacity to initiate a new job or process a partition.
     *
     * @return {@code true} if the engine has available capacity, otherwise {@code false}
     */
    boolean hasCapacity();

    /**
     * Attempts to start a job identified by {@code jobName} using the provided {@code jobParams} for job parameters.
     *
     * @param jobName   the name of the job to start.
     * @param jobParams the parameters to be used for the job execution.
     * @return an {@link Optional} containing the job instance ID if the job is successfully started or already exists;
     * an empty {@link Optional} if the job cannot be started due to invalid parameters.
     * @throws JobRestartException if the job cannot be restarted.
     * @throws NoSuchJobException  if the specified job does not exist.
     */
    Optional<Long> startJob(@NotNull final String jobName, @Nullable final JobParameters jobParams)
            throws JobRestartException, NoSuchJobException;

    /**
     * Attempts to restart a job using the specified {@code jobInstanceId}.
     *
     * @param jobInstanceId the ID of the job instance to restart.
```

```
     * @return an {@link Optional} containing the job execution ID if the job is successfully restarted or is currently running;
     * an empty {@link Optional} if the job instance does not exist.
     * @throws NoSuchJobException            if the specified job does not exist.
     * @throws NoSuchJobExecutionException   if no job execution exists for the job instance.
     * @throws JobParametersInvalidException if the job parameters are invalid for restart.
     * @throws JobRestartException           if the job cannot be restarted due to restart restrictions.
     */
    Optional<Long> restartJob(@NotNull final Long jobInstanceId)
            throws NoSuchJobException, NoSuchJobExecutionException, JobParametersInvalidException, JobRestartException;

    /**
     * Attempts to stop a job execution associated with the given {@code jobInstanceId}. It retrieves the most recent {@link JobExecution} for the given
     * {@link JobInstance} and sends a stop signal. It returns {@code true} if the stop signal was successfully sent,
     * indicating the request to stop the job was initiated but not necessarily completed.
     * <ul>
     *   <li>If the job instance does not exist, it returns {@code false}.</li>
     *   <li>If there is no job execution associated with the job instance, it returns {@code false}.</li>
     *   <li>If a valid job execution is found, it sends a stop signal and returns {@code true}. This does not mean the job has been successfully
stopped.</li>
     * </ul>
     *
     * @param jobInstanceId the ID of the job instance for which the stop operation is to be initiated.
     * @return {@code true} if the stop signal is successfully sent; {@code false} if the job instance or execution does not exist.
     * @throws NoSuchJobExecutionException     if no job execution exists for the specified job instance.
     * @throws JobExecutionNotRunningException if the job execution is not currently running.
     * @see JobOperator#stop(long) for the mechanism of stopping job executions
     */
    boolean initiateStopJob(@NotNull final Long jobInstanceId)
            throws NoSuchJobExecutionException, JobExecutionNotRunningException;

    /**
     * Provides a recovery mechanism for jobs that are stuck in the {@link BatchStatus#STOPPING} state with no active worker to complete the transition.
     * It can only be used when the job has exceeded its configured timeout period, ensuring that recovery isn't attempted prematurely.
     * <p>
     * Valid state transitions: from STOPPING to FAILED or ABANDONED.
     *
     * @param jobInstanceId the ID of the job instance to recover.
     * @param status        the new status to transition the job to, must be {@link BatchStatus#FAILED} or {@link BatchStatus#ABANDONED}.
     * @return {@code true} if the job was successfully recovered to the specified terminal state.
     * @throws NoSuchJobInstanceException if the job instance or execution does not exist.
     * @throws IllegalArgumentException   if the provided status is neither FAILED nor ABANDONED.
     */
    boolean updateJobStatus(@NotNull final Long jobInstanceId, @NotNull final BatchStatus status)
            throws NoSuchJobInstanceException, IllegalArgumentException;
}
```

Some additional notes:

- `restartJob`: Attempts to create a new job execution for an existing job instance, typically to resume processing after a failure or manual stop. It is not possible to restart a job that has already completed. In such cases, you must start a new job.
- `initiateStopJob`: A successful `initiateStopJob` does not necessarily mean the job was stopped. See Spring JobOperator for more information.
- `updateJobStatus`: A recovery can only occur when the job has exceeded its configured timeout period, specified by a system parameter, ensuring that recovery isn't attempted prematurely.

BatchEngineThrottleService

The `BatchEngineThrottleService` is responsible for determining if the batch engine has capacity to accept additional jobs based on system resource availability.

```
public interface BatchEngineThrottleService {

    /**
     * Determines whether the batch engine should accept an additional job execution.
     * <p>
     * This method evaluates the current system resources to decide if capacity is
     * available for another job execution.
     *
     * @return {@code true} if the system has capacity for another job, {@code false} otherwise
     */
    boolean shouldAcceptAdditionalJob();
}
```

The interface provides a mechanism to throttle batch job execution based on system resource availability. Implementations may use various strategies to determine capacity, such as:

- Thread pool availability
- System CPU utilization
- Available memory
- Custom resource metrics

The default implementation, `BatchEngineThrottleServiceImpl`, makes decisions based on thread pool utilization. This implementation makes a critical assumption about how batch jobs are deployed and scaled: **Projects using this service are expected NOT to spawn additional threads within their job implementations.** Instead, they should scale by adjusting the number of worker nodes with the appropriate thread count based on available machine resources.

The default throttling logic compares the current active thread count against the maximum pool size, assuming that:

- Each job consumes a predictable amount of resources
- The thread pool is configured appropriately for the host machine's capacity
- Jobs don't create their own threads that would bypass this throttling mechanism

For proper scaling with the default implementation, projects should:

1. Configure thread pool size based on available CPU cores and memory
2. Scale horizontally by adding more worker nodes rather than increasing thread count

3. Ensure jobs follow single-threaded execution patterns and only use remote partitioning for distributed processing

OnJobInstanceCreateTrigger

Functional interface to allow custom logic to run after a `JobInstance` is created.

```
/**
 * Functional interface for handling {@link org.springframework.batch.core.JobInstance} creation events.
 * <p>
 * Implementations of this interface are triggered (respecting spring {@link org.springframework.core.annotation.Order} annotation)
 * when a job instance is created,
 * allowing for custom post-creation processing or side effects.
 */
@FunctionalInterface
public interface OnJobInstanceCreateTrigger {
    /**
     * Invoked when a Spring Batch JobInstance has been successfully created for a launched job.
     * <p>
     * This callback is triggered after {@code JobLauncher#run(Job, JobParameters)} returns.
     * </p>
     * <p>
     * Typical uses include persisting the id for auditing/correlation, emitting metrics,
     * or kicking off dependent processes that need to reference the created {@code JobInstance}.
     * Implementations should be fast, non-blocking, and avoid throwing exceptions so that
     * other handlers can be notified reliably.
     * Note that depending on the {@code JobLauncher}/TaskExecutor configuration, the job may
     * still be running asynchronously when this method is called.
     * </p>
     *
     * @param jobInstanceId the unique identifier of the created {@code JobInstance}; never null
     * @see org.springframework.batch.core.JobInstance
     * @see org.springframework.batch.core.launch.JobLauncher#run(org.springframework.batch.core.Job, org.springframework.batch.core.JobParameters)
     */
    void onJobInstanceCreate(@NotNull Long jobInstanceId);
}
```

Item

[Link to item package]

The item package is responsible for all the logic concerning batch items, as well as the reading, processing, and writing of batch items. It is split into two modules, service and components. The former contains service classes for management of the `BatchItem` lifecycle, and the latter provide abstractions that should be used when implementing batch jobs. It also includes builders for creating steps, factories for generating batch jobs and step builders, as well as builders for partitioning managers and workers. The item package also contains the listener implementations.

In Spring Batch, an *item* refers to a unit of data that is read, processed, and written. The framework provides a set of abstractions to handle items efficiently. These are the `ItemReader`, `ItemProcessor`, and `ItemWriter`. There are various default implementations of these for different scenarios. For more information about these, see sections ItemReaders and ItemWriters and Item processing in the Spring Batch documentation.

Foundation Batch provides some abstractions to ease the implementation of project and business-specific logic. These are described below. By using them when creating your own batch jobs, you can easily adhere to the best practices.

Item - Service

BatchItem

`BatchItem` is the item that we use in Foundation, and it is an interface that wraps a unit of data (a "value") in a record format to enforce and encourage idempotency. A `BatchItem` contains its item's current status and has a reference to its job instance and its step. The interface is shown below.

```
/**
 * Represents a batch item with associated metadata and status tracking.
 *
 * @param <T> the record type containing the item's data
 */
public interface BatchItem<T extends Record & Serializable> {

    @NotNull
    String getId();

    @NotNull
    T getValue();

    @NotNull
    BatchItemStatus getStatus();

    @NotNull
    Long getJobInstanceId();

    @NotNull
    String getStepReference();
}
```

The value is your data and the type `T` should extend classes `Record` & `Serializable`. A simple value DTO might look like this:

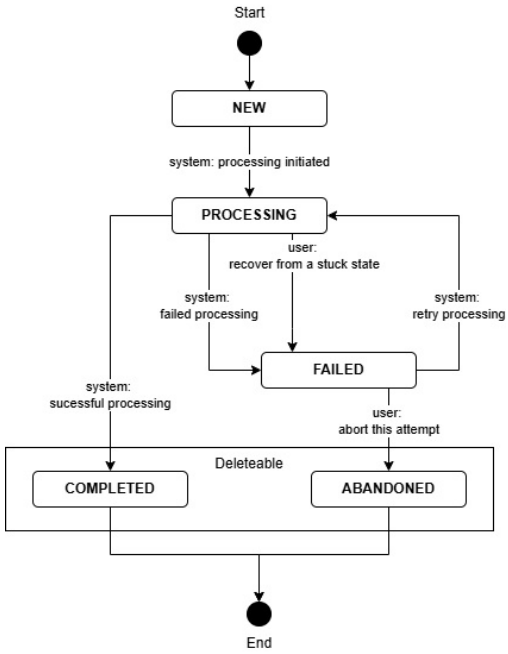
```
public record SequentialBatchJobExampleCustomerDto(
        String customerId,
        String customerName
) implements Serializable {
}
```

Status of Batch Items

The table below details the statuses a BatchItem can have.

Status	Description
`NEW`	The item has been created and is ready to be processed, but no actions have been taken on it yet.
`PROCESSING`	The item is currently being processed, indicating that it is actively undergoing the operations defined in the batch job.
`COMPLETED`	The item has been successfully processed, and all necessary operations have been carried out without errors.
`FAILED`	An error occurred during the processing of the item, preventing it from being completed successfully.
`ABANDONED`	The item has been deliberately set aside and will not be processed further, often due to business logic or irrecoverable errors.

The allowed transitions between various statuses of a batch item are displayed in the figure below.



Allowed state transitions for a batch item.

> **Note:** Only batch items with status `FAILED` or `ABANDONED` are deletable.

Batch Item Services

For an explanation of the batch item services, please see the provided JavaDoc here or in the code.

BatchItemService

The `BatchItemService` interface is responsible for managing batch items, including adding new items, updating their status, and handling deletion and recovery processes. The methods are mainly used by reader, processor, and writer components, as well as for REST operations. The service ensures that batch items transition through various statuses in a controlled manner. The allowed transitions are described in status of a batch item.

```
/**
 * Service for managing batch items: adding, reading, and updating item statuses.
 */
public interface BatchItemService {

    /**
     * Adds new batch items to the database with {@link BatchItemStatus#NEW} status.
     *
     * @param jobInstanceId the job instance ID
     * @param targetStepRef the target job step reference
     * @param values        items to add
     * @param parentId      parent item ID (optional)
     * @return list of added items
     */
    <T extends Record & Serializable> @NotNull List<BatchItem<T>> addNew(@NotNull Long jobInstanceId,
                                                                         @NotNull BatchItemStep targetStepRef,
                                                                         @NotNull List<@NotNull T> values,
                                                                         @Nullable String parentId);

    /**
     * Reads batch items from the database based on query criteria.
     *
     * @param queryCommand query criteria
     * @return list of matching items
     */
    <T extends Record & Serializable> @NotNull List<BatchItem<T>> getItems(@NotNull BatchItemQuery queryCommand);

  /**
   * Counts batch items from the database based on query criteria.
   *
   * @param queryCommand query criteria
```

```
   * @return count of matching items
   */
  long countItems(@NotNull BatchItemQuery queryCommand);

    /**
     * Updates item statuses to {@link BatchItemStatus#PROCESSING}.
     */
    void markAsProcessing(@NotNull Collection<? extends BatchItem<?>> batchItems);

    /**
     * Updates item statuses to {@link BatchItemStatus#COMPLETED}.
     */
    void markAsCompleted(@NotNull Collection<? extends BatchItem<?>> batchItems);

    /**
     * Updates item statuses to {@link BatchItemStatus#FAILED}.
     */
    void markAsFailed(@NotNull Collection<? extends BatchItem<?>> batchItems);

    /**
     * Updates item statuses to {@link BatchItemStatus#ABANDONED}.
     */
    void markAsAbandoned(@NotNull Collection<? extends BatchItem<?>> batchItems);

    /**
     * Recovers items stuck in {@link BatchItemStatus#PROCESSING} by marking them as {@link BatchItemStatus#FAILED}.
     * Only works on dormant job instances.
     *
     * @param jobInstanceId job instance ID
     * @param interJobReff  inter-job reference filter
     * @return number of items updated
     * @throws BatchException if job instance/execution invalid or job is running
     */
    int recoverProcessingItems(@NotNull Long jobInstanceId, @NotNull String interJobReff) throws BatchException;

    /**
     * Updates a batch item status to FAILED, COMPLETED, or ABANDONED.
     * Only allowed when job execution is dormant (STOPPED, FAILED, COMPLETED, or ABANDONED).
     *
     * @param batchItemId   batch item ID
     * @param updatedStatus new status (FAILED, COMPLETED, or ABANDONED)
     * @throws IllegalArgumentException if item ID invalid or status unexpected
     * @throws BatchException           if job instance/execution invalid or job not dormant
     */
    void updateItemStatus(@NotNull final String batchItemId, @NotNull final BatchItemStatus updatedStatus)
            throws IllegalArgumentException;

    /**
     * Deletes batch items by IDs. Only items from completed job instances are deleted.
     *
     * @param batchItemIds item IDs to delete
     * @return number of items actually deleted
     */
    int deleteItems(@NotNull final List<String> batchItemIds);

    /**
     * Deletes items using query criteria with custom filters.
     * Constructs a delete query based on the provided query object attributes.
     * Only items from completed job instances are deleted, others are ignored.
     * Uses a loop to ensure all matching items are deleted in one execution.
     *
     * @param query delete query criteria and filters
     * @return number of items actually deleted
     */
    int deleteItemsWithQuery(@NotNull final BatchItemBulkDeleteQuery query);
}
```

BatchItemFactory

Factory component responsible for creating batch item related entities, the factory should always be used to ensure that the proper batch item JPA is created or queried.

```
 /**
 * Factory for creating persistent batch item entities from arbitrary input values.
 */
public interface BatchItemFactory {

    /**
     * Returns the concrete {@link AbstractBatchItemJpa} type produced by this factory.
     *
     * @return the {@code Class} of the JPA entity that this factory creates; never {@code null}
     */
    Class<? extends AbstractBatchItemJpa> getBatchItemJpaClass();

    /**
     * Creates a new {@link AbstractBatchItemJpa} instance that represents the supplied value and metadata.
     * <p>
     * The created entity will:
     * - be associated with the provided job instance identifier,
     * - target the specified batch processing step,
     * - optionally reference a parent item via {@code parentId}, if provided,
     * - encapsulate the given value payload.
```

```
     * <p>
     * Nullability annotations on parameters indicate required vs. optional inputs. Implementations may
     * validate inputs and throw runtime exceptions if constraints are violated.
     *
     * @param jobInstanceId the identifier of the owning job instance; must not be {@code null}
     * @param targetStepRef the target step this item is intended for; must not be {@code null}
     * @param parentId      optional identifier of a parent item, if this item is part of a hierarchy; may be {@code null}
     * @param value         the payload to wrap into a batch item; must not be {@code null}; must be a {@link Record} and {@link java.io.Serializable}
     * @param <T>           the payload type, constrained to Java records that are also {@link java.io.Serializable}
     * @return a new, non-null {@link AbstractBatchItemJpa} representing the given inputs
     * @throws NullPointerException     if any parameter annotated {@code @NotNull} is {@code null}
     * @throws IllegalArgumentException if the inputs are otherwise invalid for the concrete implementation
     */
    <T extends Record & Serializable> AbstractBatchItemJpa createBatchItem(
            @NotNull Long jobInstanceId,
            @NotNull BatchItemStep targetStepRef,
            @Nullable String parentId,
            @NotNull T value
    );
}
```

BatchItemRepository

The `BatchItemRepository` is an interface acting as a data access component responsible for performing operations on batch items stored in the database. Its primary responsibilities include retrieving batch items, item lookup, deletion operations, and query-based operations. `getItems` is used to retrieve the batch items for a specific job instance and step in `ChunkLoadingBatchItemReader`, see section BatchItemReader.

```
/**
 * Repository for batch item data access operations.
 */
public interface BatchItemRepository {

    /**
     * Retrieves a batch item by its ID.
     *
     * @param batchItemId the batch item ID
     * @return optional containing the item if found
     */
    Optional<BatchItem> getItemById(@NotNull String batchItemId);

    /**
     * Retrieves batch items based on query criteria.
     *
     * @param queryCommand query criteria and filters
     * @return list of matching batch items
     */
    <T extends Record & Serializable> @NotNull List<BatchItem<T>> getItems(@NotNull BatchItemQuery queryCommand);

    /**
     * Deletes batch items by their IDs. Only items from completed or abandoned job instances are deleted.
     *
     * @param batchItemIds list of item IDs to delete
     * @return number of items actually deleted
     */
    int deleteItems(@NotNull List<String> batchItemIds);

    /**
     * Deletes batch items using query criteria. Only items from completed or abandoned job instances are deleted.
     * Uses pagination to handle large datasets efficiently.
     *
     * @param query delete query criteria and filters
     * @return number of items actually deleted
     */
    int deleteItemsWithQuery(@NotNull BatchItemBulkDeleteQuery query);

    /**
     * Counts the total number of batch items stored in the repository.
     *
     * @return the total count of batch items
     */
    long countBatchItems();

    /**
     * Counts the number of batch items matching the specified query criteria.
     *
     * @param queryCommand the query criteria and filters
     * @return the count of matching batch items
     */
    long countBatchItems(BatchItemQuery queryCommand);

    /**
     * Creates and persists a new batch item for the given job instance and step.
     *
     * @param jobInstanceId the ID of the job instance to associate with this item
     * @param targetStepRef the target batch step reference
     * @param parentId      optional parent batch item ID (may be {@code null})
     * @param value         the data value to store in the batch item
     * @param <T>           the type of the batch item value (must be {@link Record} and {@link Serializable})
     * @return the persisted {@link BatchItem}
     */
    <T extends Record & Serializable> BatchItem<T> createAndPersistBatchItem(
```

```
            @NotNull Long jobInstanceId,
            @NotNull BatchItemStep targetStepRef,
            @Nullable String parentId,
            @NotNull T value
    );

    /**
     * Updates the status of batch items in bulk according to the criteria specified in the query command.
     *
     * <p>
     * The selection and update of batch items are determined by the filters and target status defined in
     * the {@link BatchItemBulkUpdateStatusQuery}. This operation is typically used to efficiently update
     * the status of multiple batch items in a single call.
     * </p>
     *
     * @param queryCommand the bulk update query containing selection criteria and the new status to apply (must not be {@code null})
     * @return the number of batch items whose status was updated
     */
    int updateBatchItemStatus(BatchItemBulkUpdateStatusQuery queryCommand);
}
```

Item Error Handling

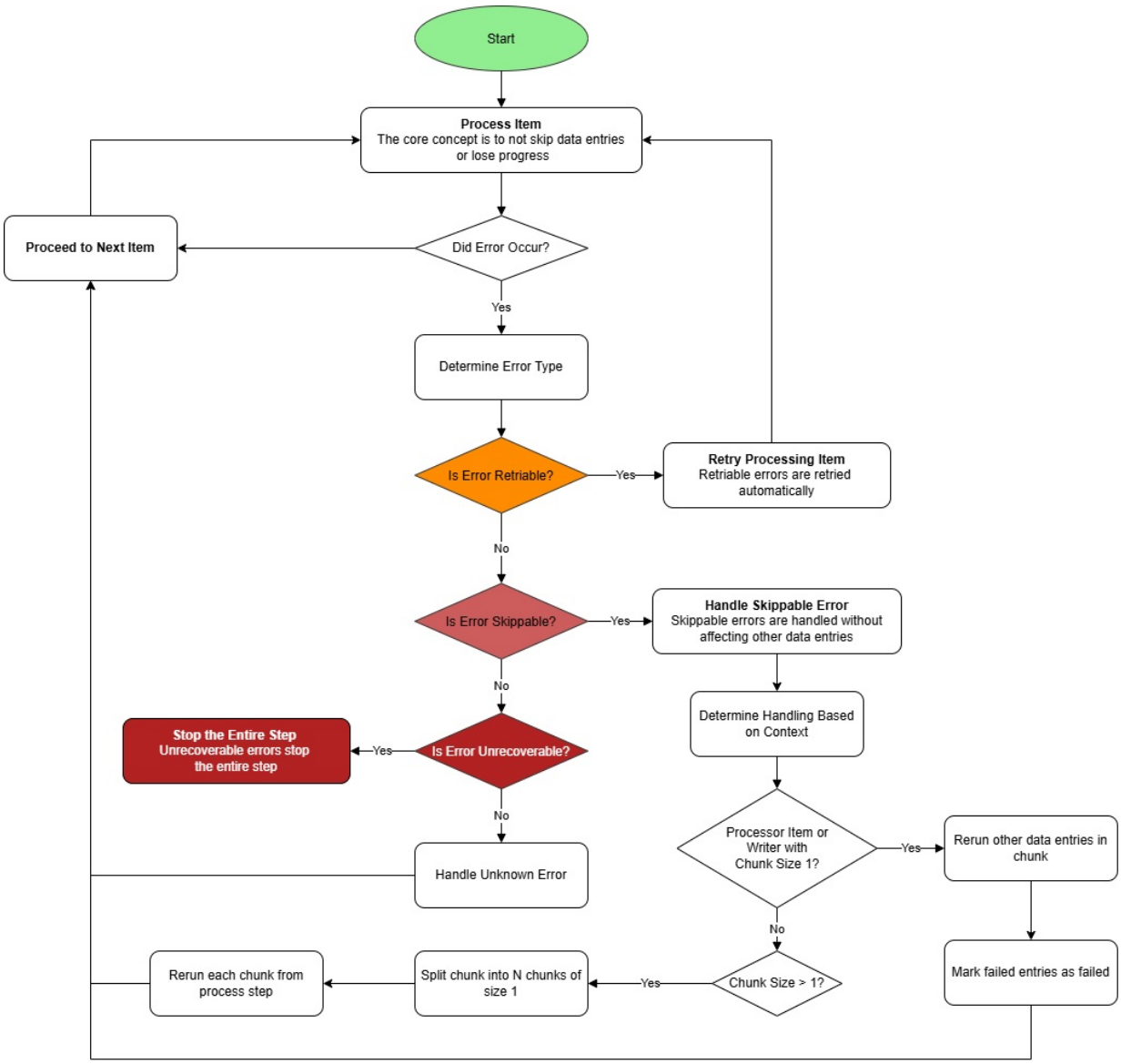
Foundation Batch has a strategy for how to retry failed items within the context of its instance.

The core concept of error handling in Foundation Batch is that it should not be possible to skip a data entry or lose track of progress. If an item fails, it is important that it is always marked as failed and it should not affect other data entries. The way errors are handled can vary and there are three degrees of error:

- **Retryable**: Retry the process and see if it is successfully run.
- **Skippable**: This error can be resolved in two different ways.
  - In the processor for all chunk sizes or writer when the chunk size is 1: rerun all the other data entries in the chunk. The one(s) that failed are marked as failed so they can be handled manually, investigated, or potentially just rerun.
  - In the writer when the chunk size is greater than 1: The chunk will be split into N many chunks of size 1. Then each chunk is rerun from the process step.
- **Unrecoverable**: Stop the entire step because something has gone very wrong.

It is important to be aware that when rerunning a failed step, there may be artifacts left from the previous iteration. The Foundation Batch framework strives to enforce purity, meaning that a step or item with the same input should consistently produce the same output. While it is possible to create impure batch jobs, doing so can lead to significant issues. For more information on maintaining purity and avoiding common pitfalls, please refer to the best practices.

Diagram for Error Handling



Item - Components

BatchItemComponentsConfig

A Spring configuration class that provides default beans for batch item processing components. These are reusable components that handle common batch operations for item-based processing.

Its purpose is to:

- Provide step-scoped default implementations for readers, writers, partitioners, and listeners
- Enable component scanning for batch item components

Bean	Type	Purpose	Scope
`chunkLoadingBatchItemReader`	`ChunkLoadingBatchItemReader<T>`	Reads batch items in chunks for processing	Step
`batchItemCreationWriter`	`BatchItemCreationWriter<I>`	Creates new batch items during processing	Step
`simpleBatchItemCompletionWriter`	`SimpleBatchItemCompletionWriter<I, O>`	Marks batch items as completed after processing	Step
`simpleBatchItemFollowupWriter`	`SimpleBatchItemFollowupWriter<I, O>`	Handles follow-up actions for processed batch items	Step
`batchItemRangePartitioner`	`BatchItemRangePartitioner`	Partitions batch items by range for parallel processing	Step
`failOnCompletedWithFailedItemsJobExecutionListener`	`FailOnCompletedWithFailedItemsJobExecutionListener`	Fails job execution when items have failed	Singleton
`baseBatchItemSkipListener`	`BaseBatchItemSkipListener`	Handles skipped items during batch processing	Step
`batchItemStateGuardDecider`	`BatchItemStateGuardDecider`	Verifies the state of existing batch items related to the job before processing	Singleton

Reader

In Spring Batch, the ItemReader is a crucial component that helps in reading data from a specified source at the beginning of a batch job. Its primary purpose is to abstract the details of data retrieval from various types of data sources, such as databases, files, queues, etc., and to provide a consistent interface for reading data in chunks.

In Foundation, we differentiate between two types of readers. The first one is used in the create items step of a batch job. It uses the readers from Spring Batch directly and reads from any data source. Spring provides readers for reading data in various formats.

The other type has the purpose of reading batch items, done in subsequent steps of a batch job.

BatchItemReader

`BatchItemReader` extends the Spring Batch interface `ItemReader` and is a wrapper to enforce the BatchItem rules on the reader. An implementation of this interface needs to implement the `read()` method, where some data should be read from a data source somewhere and return a batch item with the data.

```
/**
 * Reader for batch items that extends Spring Batch's ItemReader, requiring the read() method.
 * Provides sequential access to BatchItem instances during batch processing.
 *
 * @param <T> the type of data contained in the batch items (must be Record and Serializable)
 */
public interface BatchItemReader<T extends Record & Serializable> extends ItemReader<BatchItem<T>> {
}
```

Class Name	Description
`ChunkLoadingBatchItemReader`	The default chunk-reader implementation of `BatchItemReader`, which allows for reading chunks of batch items efficiently. It uses synchronization methods to enable usage in a multi-threaded environment. > Using this reader is sufficient for most use cases.

Processor

In Spring Batch, the ItemProcessor is a component that processes data between reading and writing phases in a batch job. The primary purpose of an ItemProcessor is to transform or filter the data that has been read before it is written to the output. Given one object, transform it and return another. The provided object may or may not be of the same type. Hence, this is where business-specific logic can be implemented.

> **Important:** It is very important that the processing step is idempotent, i.e., re-running the processing step with the same batch items **should run exactly the same and yield the same output**.

BatchItemProcessor

The `BatchItemProcessor` encapsulates the method `process` from Spring's ItemProcessor. The interface provides a default method for processing `BatchItems` using its methods `processBatchItem` and the `getBatchItemContext`. A `BatchItemContext` is created when processing a batch item. It holds the input batch item together with the resulting items after processing. These can later be accessed from the context in the writer.

```
/**
 * Processor for transforming batch items from input type to output type.
 * Extends Spring Batch's ItemProcessor to work with BatchItem and BatchItemContext.
 *
 * @param <I> input item type (must be Record and Serializable)
 * @param <O> output item type (must be Record and Serializable)
 */
public interface BatchItemProcessor<I extends Record & Serializable, O extends Record & Serializable>
        extends ItemProcessor<BatchItem<I>, BatchItemContext<I, O>> {
```

```
    /**
     * Processes a batch item by transforming it and wrapping the result in a context.
     * Default implementation delegates to processBatchItem() and getBatchItemContext().
     *
     * @param item the batch item to process
     * @return context containing the input item and processed output items
     * @throws Exception if processing fails
     */
    default BatchItemContext<I, O> process(@NotNull BatchItem<I> item) throws Exception {
        return getBatchItemContext(item, processBatchItem(item));
    }

    /**
     * Creates a batch item context from the input item and processing results.
     *
     * @param item                the original input batch item
     * @param itemProcessingOutput the processed output items
     * @return context wrapping input and output items
     */
    @NotNull
    BatchItemContext<I, O> getBatchItemContext(@NotNull BatchItem<I> item,
                                               @NotNull List<@NotNull O> itemProcessingOutput);

    /**
     * Processes a single batch item and transforms it to output items.
     * Core processing logic to be implemented by concrete processors.
     *
     * @param oldItem the batch item to process
     * @return list of processed output items
     */
    @NotNull
    List<@NotNull O> processBatchItem(@NotNull BatchItem<I> oldItem);
}
```

It is in the method `processBatchItem` that the main processing of your batch job occurs. It applies some business rules to the input item `I` and returns a list of new items of the type `O`. Depending on the logic, the output and input items can share the same type. For example, if you're filtering data in your processing step, you can most likely use the same type for both. But if you're transforming some input object into a new object, you need to use different types.

The table below describes the other implementations of processors in Foundation.

Class Name	Description
`AbstractBatchItemProcessor`	A simple abstraction of the `BatchItemProcessor` that provides a framework for mapping a batch item into one or more outputs with default skip logic. > Suitable for most batch jobs. Extend this class and implement the `processBatchItem()` method.
`SimpleBatchItemProcessor`	A basic processor that applies no business logic during the processing step.

Writer

In Spring Batch, the ItemWriter is an interface responsible for writing processed data to a specified destination. This destination could be a database, a file, a message queue, or any other storage system. The writer is the final component of a step.

BatchItemWriter

In Foundation Batch, the `BatchItemWriter` is an interface that extends the Spring Batch interface `ItemWriter`. It provides a `write` method calling three functions which can be overridden: One to perform any side effects (if there are any), one to create new batch items if needed, and one to handle the batch items.

```
/**
 * Writer for batch item processing that extends Spring Batch's ItemWriter.
 * Handles writing BatchItemContext instances with support for side effects,
 * follow-up item creation, and batch item management.
 *
 * @param <I> input item type (must be Record and Serializable)
 * @param <O> output item type (must be Record and Serializable)
 */
public interface BatchItemWriter<I extends Record & Serializable, O extends Record & Serializable>
        extends ItemWriter<BatchItemContext<I, O>> {

    /**
     * Writes a chunk of batch item contexts using a three-phase approach:
     * 1. Performs side effects (external operations)
     * 2. Creates follow-up batch items if needed
     * 3. Handles batch item status updates and persistence
     *
     * @param chunk the chunk of batch item contexts to write
     * @throws Exception if any phase of writing fails
     */
    default void write(@NotNull Chunk<? extends BatchItemContext<I, O>> chunk) throws Exception {

        performSideEffect(chunk);

        handleFollowupBatchItemCreation(chunk);

        handleBatchItems(chunk);
    }

    /**
     * Handles batch item management including status updates and persistence.
```

```
     * Typically marks items as completed or failed based on processing results.
     */
    void handleBatchItems(@NotNull Chunk<? extends BatchItemContext<I, O>> chunk);

    /**
     * Creates follow-up batch items based on processing results.
     * Used when processing one item generates additional items for subsequent processing.
     */
    void handleFollowupBatchItemCreation(@NotNull Chunk<? extends BatchItemContext<I, O>> chunk);

    /**
     * Performs side effects such as external data storage, system calls, notifications, or logging.
     * Executed first in the writing process before item management.
     */
    void performSideEffect(@NotNull Chunk<? extends BatchItemContext<I, O>> chunk);
}
```

The table below describes the writer implementations available in Foundation.

Class Name	Description
`AbstractBatchItemWriter`	Provides a default `handleBatchItems()` method that marks all batch items as completed. > Suitable for most batch jobs. Use `handleFollowUpBatchItemCreation()` to create additional batch items, and `performSideEffect()` to persist data from batch items to a data source or perform other side effects as required.
`BatchItemCreationWriter`	Responsible for the initial creation and persistence of batch items using the `BatchItemService`. > Use this writer in the first step of your batch jobs (the create batch items step).
`SimpleBatchItemCompletionWriter`	A simple implementation used when there is no side effect and batch items should only be marked as completed.
`SimpleBatchItemFollowupWriter`	Default implementation used when there is no side effect, existing items should be marked as completed, and new follow-up or additional batch items should be created.

Listeners

In Spring Batch, a listener is a callback mechanism that allows you to intercept and manage the lifecycle events of a batch job, step, or chunk. Listeners can be used to execute code before or after a job starts, when a step completes, if an error occurs, or at other defined points in the process, such as when an item is read, processed, or written. They are useful for logging, monitoring, resource management, and custom error handling without directly altering the job or step configurations.

The `SkipListener` can be used to implement some fail or skip logic in case something goes wrong with a data entry in a step, instead of either hard crashing or rerunning the entire job. `StepExecutionListener` and `JobExecutionListener` handle the state of the step and job execution respectively. They can be used for monitoring, logging, or updating statuses during executions. For more information, see the Spring Batch documentation on intercepting step execution.

Listeners in Foundation

Foundation has several predefined listeners that use the interfaces provided from Spring. Most are used internally, such as `BatchItemStepConfigurationListener`, `FailOnCompletedWithFailedItemsJobExecutionListener` and `BatchItemSkipListener`.

A complete list of custom listeners in Foundation Batch can be seen in the table below.

Name	Logic
All default listeners from Spring Batch	See Intercepting Spring Step Execution.
`ContextInformationProviderJobExecutionListener`	Default listener that reads the username and tenant id on the Spring Security Context and and add the information to the `JobExecutionContext`. Used in conjunction with `ContextProvidingStepExecutionListener`
`ContextProvidingStepExecutionListener`	Default listener that reads the username and tenant id from the `JobExecutionContext` and uses this to set the Spring Security Context for the step. If a tenant is provided in the `StepExecutionContext` this will be used instead. The new context is set in the `beforeStep` method and reset to the previous context in the `afterStep` method.
`BatchItemSkipListener`	Interface for skip listeners.
`BaseBatchItemSkipListener`	Default skip listener, should be supplied in every step configuration. Implements skip logic from Item Error handling.
`BatchItemStepConfigurationListener`	Listener that configures the step context with the necessary information for the batch item step to work. This listener is used internally in the Foundation Batch.
`BatchItemRangePartitioner`	Default listener that handles partitioning of a step in a batch job.
`BatchItemTenantPartitioner`	Listener that handles partitioning of a step in a batch job by partitioning the batch items on the unique tenant ids that are present on the batch items for the step. The tenant id is put on the `StepExecutionContext`, and in combination with the the `ContextProvidingStepExecutionListener` this means that the reader, writer and processor in the step will run as the tenant provided by the partition.
`FailOnCompletedWithFailedItemsJobExecutionListener`	Default listener for setting the status of a `JobExecution` to failed if any batch items failed. Applied by default.
`AbstractStepAdvanceBlockListener`	Abstract listener to block at the end of a step, if specified condition failed it will set the status to `FAILED`, and terminate the step thus blocking the job from continue to the next step.
`AbstractStepStartBlockListener`	Abstract listener to block a step from starting, if specified condition failed it will set the status to `FAILED` and terminate the step as soon as possible.

Adding Custom Listeners

Custom listeners provide a powerful way to extend batch processing functionality while maintaining clean separation of concerns. Rather than embedding custom logic directly within e.g., the process steps, listeners offer several key advantages:

- Flexibility: Easy to modify where and when the listener triggers
- Reusability: Same listener can be applied across different steps or jobs

- Maintainability: Cleaner code organization and easier testing

Custom listeners can be seamlessly integrated into your batch steps using the `addListener()` method on step builders:

```
@Bean
public Step myStep() {
    return stepBuilderFactory.getProcessingStep(MY_STEP)
            .setReader(myReader)
            .setProcessor(myProcessor)
            .setWriter(myWriter)
            .addListener(myCustomListener)
            .build();
}
```

Foundation Batch leverages BatchStepListenerUtils internally to ensure proper listener registration. This utility performs casting to register listeners with their appropriate specific interfaces (such as StepExecutionListener, ItemReadListener, etc.).

This approach is essential because Spring Batch maintains distinct registration pathways for:

- Each of the listener interfaces: Direct implementation of listener interfaces
- Annotation-based listeners: Methods decorated with annotations like @BeforeStep, @AfterStep

By handling this casting automatically, Foundation Batch ensures your custom listeners are properly recognized and executed within the Spring Batch framework, regardless of their implementation approach.

JobExecutionDecider

The JobExecutionDecider is part of flow control for the batch job in Spring Batch. In the provided builder `DirectedPathBatchJobBuilder`, the foundation standard `BatchItemStateGuardDecider` is provided by default. Additional checks can be provided via the builders `guardDecider` method. **Guards are evaluated before any steps execute.**

Guard Decider Evaluation Logic

Registered guard conditions are executed following a fail-fast semantics:

- **Sequential Evaluation**: Guards are evaluated in the order they were added
- **Fail-Fast**: If any guard returns FAILED or STOPPED, evaluation stops immediately and that status is returned
- **Continue on Success**: Only COMPLETED status allows evaluation to proceed to the next guard

Execution Examples:

```
Guards: [COMPLETED, COMPLETED, COMPLETED] → Result: COMPLETED
Guards: [COMPLETED, FAILED, COMPLETED]   → Result: FAILED (stops at second guard)
Guards: [STOPPED, COMPLETED, COMPLETED]  → Result: STOPPED (stops at first guard)
```

BatchItemStateGuardDecider

The default provided jobExecutionDecider will check that any batch item associated with the job on which the decider has been applied, is not in the "processing" state. This is intended to ensure that any batch item that is related to the job being executed is in a permitted state prior to execution of the jobs processing steps. If a batch item with status "processing" is detected, it will cause the job to fail and abort further processing.

The item package contains a number of builders and factories for creating batch jobs and steps. The builders extend the abstract builders introduced in the engine section abstract job and step builders.

> **Note:** You need to include `BatchItemJobBuilderFactory` and `BatchItemStepBuilderFactory` as dependencies in your batch job configuration files. They allow you to create your jobs and steps.

`BatchItemJobBuilderFactory` is the factory for creating batch jobs. It allows you to create a batch job:

Class name	Description	Factory Method
`DirectedPathBatchJobBuilder`	Builder for constructing batch jobs with directed path execution flow	`getDirectedPathJob()`

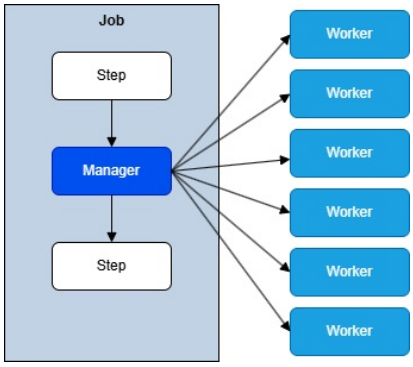
`BatchItemStepBuilderFactory` is a factory for creating various types of steps for a batch job. It allows you to create the following batch steps:

Class name	Description	Factory Method
`BatchCreationStepBuilder`	Builder for creating steps that generate and persist new batch items	`getCreationStep()`
`BatchProcessingStepBuilder`	Builder for creating general steps that process existing batch items through transformation logic	`getProcessingStep()`
`BatchPartitionManagerStepBuilder`	Builder for creating partition manager steps that coordinate parallel processing across workers	`getPartitionManagerStep()`
`BatchPartitionWorkerStepBuilder`	Builder for creating partition worker steps that execute processing tasks in parallel partitions	`getPartitionWorkerStep()`

Partitioning

Partitioning is a method that distributes the workload of a step across multiple machines, thereby enhancing scalability and performance. The logic for partitioning is partially implemented in the `BatchPartitionConfig` within the engine. This configuration is designed to enable the distribution of batch job steps across multiple worker nodes for concurrent processing.

Under the hood, the partitioning is done by Spring Batch. Spring Batch handles the partitioning process by establishing a manager step and several worker steps. The manager step configures the partitioning logic and is responsible for dividing the data and distributing it to the worker steps. These worker steps then process the data in parallel. Communication between the manager and worker steps occurs through a message queue called `partitionChannel`, which is defined in the engine's configuration file. The `partitionWorkerConsumer` plays a crucial role by consuming messages from `partitionChannel`, triggering the execution of worker steps upon receiving partitioned data. The `partitionWorkerConsumer` ensures that partitions consume worker threads from the same pool as the rest of the engine.



Partitioning overview.

Configuring Partitioning

The following section details how to configure partitioning in a batch job. For an example of a partitioned batch job, see the partitions batch job example. The code excerpts below are from this batch job.

> **Note:** If your job is not idempotent, you should not use partitioning as it can lead to data corruption.

Partitioning can easily be configured in the configuration file of a batch job. A manager step and a worker step have to be configured using `BatchItemStepBuilderFactory`.

`getPartitionManagerStep` creates a manager step using a `BatchPartitionManagerStepBuilder`. On this object, one can specify the grid size (i.e., the number of separate step executions to create) and what partitioner to use. For most use cases, you should use the default partitioner implementation in Foundation unless some very specific partitioning logic is required. An example of a manager step:

```
@Bean
public Step partitionsExampleBatchJobPartitioningOfStep(
        // ...
        BatchItemRangePartitioner batchItemRangePartitioner
) {
    return stepBuilderFactory.getPartitionManagerStep(PARTITIONS_BATCH_JOB_EXAMPLE_PARTITIONS_STEP)
            .setPartitioner(batchItemRangePartitioner)
            .setGridSize(PARTITIONS_BATCH_JOB_EXAMPLE_PARTITION_GRID_SIZE)
            .setWorkerStepBeanName(PARTITIONS_BATCH_JOB_EXAMPLE_PARTITION_WORKER_NAME)
            .build();
}
```

> **Default partitioner:** The default partitioner implementation is called `BatchItemRangePartitioner`. The partitions are decided by evenly dividing the total number of processable batch items (NEW or FAILED) into chunks based on the specified grid size, and using the item IDs as range boundaries. Each partition is assigned an `ExecutionContext` which contains the range of IDs that the partition should process. Each partition might not get the same number of items to process, and it is not guaranteed that they finish at the same time.

`getPartitionWorkerStep` creates a worker step using a `BatchPartitionWorkerStepBuilder` and this is the step which gets executed in parallel. Each worker step is configured with its own reader, processor, and writer as you would normally when creating a step that is not partitioned. An example of a worker step:

```
@Bean
@Lazy(false) // Needed to ensure that the step bean is registered upon application startup
public Step partitionsExampleBatchJobPartitioningOfStepWorker(
        //...
        ChunkLoadingBatchItemReader<BatchJobExampleLogDto> chunkLoadingBatchItemReader,
        PartitionsBatchJobExampleItemProcessor partitionsBatchJobExampleItemProcessor,
        PartitionsBatchJobExampleItemWriter partitionsBatchJobExampleItemWriter
) {
    return stepBuilderFactory.<BatchJobExampleLogDto, BatchJobExampleLogDto>getPartitionWorkerStep(
                    PARTITIONS_BATCH_JOB_EXAMPLE_PARTITIONS_STEP)
            .setChunkSize(PARTITIONS_BATCH_JOB_EXAMPLE_PARTITION_STEP_CHUNK_SIZE)
            .setReader(chunkLoadingBatchItemReader)
            .setProcessor(partitionsBatchJobExampleItemProcessor)
            .setWriter(partitionsBatchJobExampleItemWriter)
            .build();
}
```

Aggregation

Foundation's batch aggregation framework addresses the common need to consolidate multiple batch items into fewer aggregated results - implementing the "reduce" part of map-reduce pattern for batch processing. This fills a gap where you need to group, merge, and transform collections of items rather than processing them individually.

The pattern employed is one where items are transformed into the aggregate representation with a sub set identifier, an *aggregation key*. Aggregates with the same aggregation key are then merged. As this process is done in a distributed manner, no guarantees about order can be made.

The framework provides structured components for this aggregation pattern utilizing and extending the Spring Batch Framework.

As for `BatchItem`, the services are placed in the module `item-service` and the components for use in batch jobs in `item-components`.

AbstractBatchItemAggregator

In Foundation Batch, the `AbstractBatchItemAggregator` is an abstract class that extends Spring Batch's `DefaultStepExecutionAggregator`. It provides a framework for aggregating partition worker results, merging duplicate items by aggregation key, and creating output items for downstream steps using the `AggregateItemMerger` for business logic.

```
/**
 * Abstract partition manager aggregator for merging worker step results in Spring Batch.
 *
 * Extends DefaultStepExecutionAggregator to handle aggregation of partition worker results,
 * merge duplicate items by aggregation key, and create output items for downstream steps.
 * Uses AggregateItemMerger for business logic.
```

```
 *
 * @param <I> Input record type (Record & Serializable)
 * @param <A> Aggregate record type (Record & Serializable)
 * @param <O> Output record type (Record & Serializable)
 */
public abstract class AbstractBatchItemAggregator<
                I extends Record & Serializable,
                A extends Record & Serializable,
                O extends Record & Serializable
                > extends DefaultStepExecutionAggregator {

    /**
     * Aggregates partition worker step results by merging items and creating output for target steps.
     *
     * <p>Processing flow:
     * <ol>
     *   <li>All aggregation keys currently in use</li>
     *   <li>For each key, merge partial aggregates</li>
     *   <li>Deletes existing aggregates</li>
     *   <li>If aggregations is not completed, persist current results.</li>
     *   <li>If aggregations is completed, create output items for target steps.</li>
     * </ol>
     *
     * @param result     The manager step execution result
     * @param executions Collection of worker step executions to aggregate
     */

    @Override
    public void aggregate(@NonNull StepExecution result, @NonNull Collection<StepExecution> executions);

    /**
     * Determines if aggregation is complete and batch items should be pushed to target steps.
     * Must be implemented by concrete classes to define completion criteria.
     */
    protected abstract boolean aggregationFinished(StepExecution result, Collection<StepExecution> executions);
}
```

The aggregator follows a template method pattern where concrete implementations define when aggregation is finished, while default implementations handle common querying patterns. Subclasses can override the data retrieval methods for custom aggregation strategies.

AggregateItem

`AggregateItem` is the item that we use in Foundation for aggregation workflows, and it is an interface that wraps aggregated data in a record format to support batch processing with consolidation capabilities. An `AggregateItem` contains the aggregated value, metadata for tracking its lifecycle through aggregation steps, and a key for grouping related items. The interface is shown below.

```
/**
 * Represents an aggregate item in batch processing with associated metadata.
 *
 * @param <A> Aggregate record type (Record & Serializable)
 */
public interface AggregateItem<A extends Record & Serializable> {

    @Nullable
    String id();

    @NotNull
    Long jobInstanceId();

    @NotNull
    String creatorStepReference();

    @NotNull
    Long stepExecutionId();

    @NotNull
    String aggregationKey();

    @NotNull
    A value();

    AggregateItem<A> withId(@NotNull String newId);
}
```

The value is your aggregated data and the type `A` should extend classes `Record` & `Serializable`. The `aggregationKey` is used to group related items for consolidation during the aggregation process. Items with the same aggregation key will be merged together using business-specific logic. A simple aggregate value DTO might look like this:

```
public record AggregateLogDto(
        String message,
        String level,
        Long count
) implements Serializable {
}
```

The `stepExecutionId` tracks which partition worker created the item, while `creatorStepReference` identifies the processing step. The `withId()` method provides immutable updates for ID assignment after persistence.

AggregateItemService

The `AggregateItemService` interface is responsible for managing aggregate items.

```java
package nc.foundation.libraries.batch.item.service.api;

import jakarta.annotation.Nullable;
import jakarta.validation.constraints.NotNull;
import nc.foundation.libraries.batch.item.service.service.api.types.AggregateItem;
import nc.foundation.libraries.batch.item.service.service.api.types.AggregateItemQuery;
import nc.foundation.libraries.batch.item.service.service.api.types.BatchItemStep;

import java.io.Serializable;
import java.util.List;

/**
 * Service interface for managing aggregate items in batch processing workflows.
 *
 * <p>Provides CRUD operations for aggregate items including creation, persistence,
 * querying, and deletion. Supports batch job metadata tracking and step execution context.
 */
public interface AggregateItemService {

    /**
     * Persists a list of aggregate items to storage.
     *
     * @param aggregateItems List of aggregate items to persist
     * @param <A>            Aggregate record type (Record & Serializable)
     * @return List of persisted aggregate items with updated metadata
     */
    <A extends Record & Serializable> @NotNull List<AggregateItem<A>> persist(
            @NotNull List<AggregateItem<A>> aggregateItems);

    /**
     * Creates a new aggregate item with batch job metadata.
     *
     * @param id                   Unique identifier for the item (null for auto-generation)
     * @param jobInstanceId        Batch job instance identifier
     * @param creatorStepReference Reference to the step that created this item
     * @param stepExecutionId      Step execution identifier (null for final results)
     * @param aggregationKey       Key used for grouping related items
     * @param value                The aggregate data payload
     * @param <A>                  Aggregate record type (Record & Serializable)
     * @return Created aggregate item with metadata
     */
    <A extends Record & Serializable> AggregateItem<A> createAggregateItem(
            @Nullable String id,
            @NotNull Long jobInstanceId,
            @NotNull BatchItemStep creatorStepReference,
            @NotNull Long stepExecutionId,
            @NotNull String aggregationKey,
            @NotNull A value
    );

    /**
     * Retrieves aggregate items based on query criteria.
     *
     * @param queryCommand Query parameters for filtering items
     * @param <A>          Aggregate record type (Record & Serializable)
     * @return List of matching aggregate items
     */
    <A extends Record & Serializable> @NotNull List<AggregateItem<A>> getItems(
            @NotNull AggregateItemQuery queryCommand);

    /**
     * Retrieves aggregate keys based on query criteria.
     *
     * @param queryCommand Query parameters for filtering items
     * @return List of matching aggregate keys
     */
    @NotNull
    List<String> getKeys(@NotNull AggregateItemQuery queryCommand);

    /**
     * Deletes aggregate items by their identifiers.
     *
     * @param aggregateItemIds List of item IDs to delete
     * @return Number of items successfully deleted
     */
    int deleteItemsById(@NotNull List<String> aggregateItemIds);
}
```

AggregateItemMerger

The `AggregateItemMerger` is a key component used by both the `AbstractBatchItemAggregator` and processors to handle the business logic of merging and transforming aggregate items. It provides a multi-stage transformation pipeline for processing batch items through aggregation workflows.

```java
/**
 * Interface for merging and aggregating batch items through a transformation pipeline.
 *
 * <p>Processes items through: Input → Aggregate → Aggregation → Output
 *
 * @param <I> Input record type (Record & Serializable)
 * @param <A> Aggregate record type (Record & Serializable)
 * @param <O> Output record type (Record & Serializable)
```

```
 */
public interface AggregateItemMerger<
                I extends Record & Serializable,
                A extends Record & Serializable,
                O extends Record & Serializable
                > {

    /**
     * Wraps aggregate data with batch job metadata.
     */
    AggregateItem<A> fromItem(A aggregate, Long jobInstanceId, String creatorStepReference, Long stepExecutionId);

    /**
     * Converts input BatchItem to aggregate format.
     */
    A convertFromInput(BatchItem<I> input);

    /**
     * Transforms aggregate items to final output format.
     */
    List<O> toOutputItems(List<AggregateItem<A>> aggregateItems, String stepReference);

    /**
     * Processes and consolidates aggregate items by grouping and merging.
     *
     * @param stepExecution Current step execution context
     * @param items Items to aggregate
     * @return Processed aggregate items
     */
    List<AggregateItem<A>> aggregate(
            StepExecution stepExecution,
            List<AggregateItem<A>> items
    );
}
```

The `AggregateItemMerger` handles four key responsibilities:

1. **Input Conversion** - Transforms input batch items to aggregate format
2. **Metadata Wrapping** - Adds batch processing metadata to aggregates
3. **Aggregation Logic** - Groups and merges items by aggregation key
4. **Output Transformation** - Converts final aggregates to output format

The merger is used in two main contexts:

- **By Aggregators** - For consolidating partition worker results and creating output for downstream steps
- **By Processors** - For converting individual batch items to aggregate format during processing

The table below describes the merger implementations and related components available in Foundation.

Class Name	Description
`AbstractAggregateItemMerger`	Provides default implementations for metadata handling and aggregation orchestration. Requires concrete implementations to define aggregation key calculation, merge logic, and output conversion. > Suitable for most aggregation scenarios. Implement `calculateAggregationKey()`, `merge()`, and `convertToOutput()` methods to define your business logic.
`SimpleAggregateItemProcessor`	A processor that converts batch items to aggregate format using an `AggregateItemMerger`. Uses only the `convertFromInput()` method to transform individual items. > Use this processor in partition worker steps to transform input items to aggregate format for subsequent aggregation.
`AbstractAggregateItemWriter`	A writer that extends batch item writing with aggregate item creation and persistence. Uses the merger to create aggregate items from processed output and persists them for partition manager aggregation. > Use this writer in partition worker steps to create and persist aggregate items alongside regular batch item processing.

Scheduler

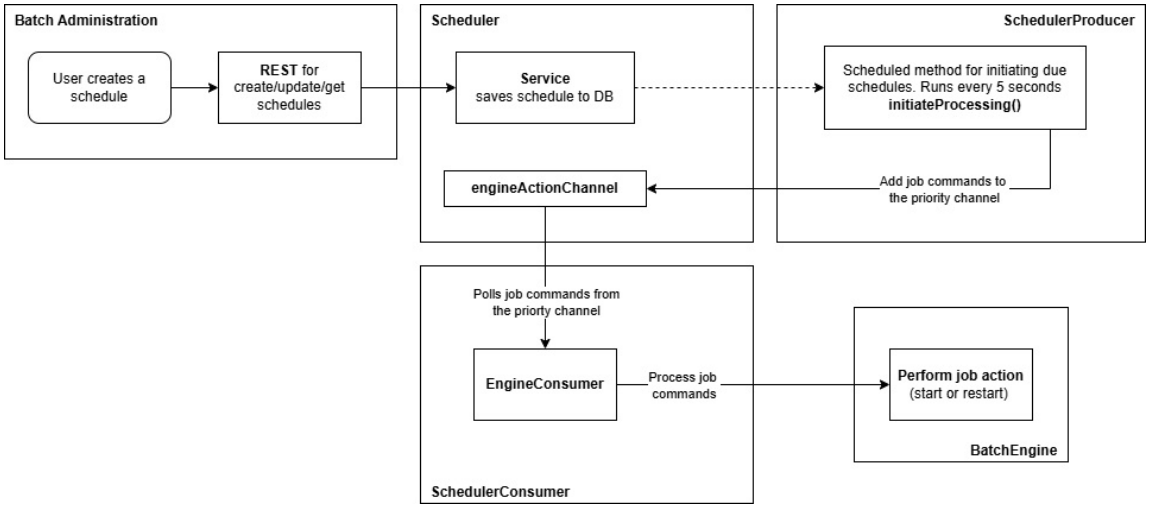
Link to scheduler package

The scheduler package manages schedules, and determines when a job should be run. The scheduler is divided into three parts: service, producer and consumer. The service contains logic for management of schedules. The producer initiates batch job schedules that are due for execution at a specified time by producing job commands. The job commands are passed to a PriorityChannel (engineActionChannel) bean inside the service package. The consumer then polls messages from the channel and processes the job commands based on its type.

Each batch job instance has its own associated schedules. Schedules are created using CRON expressions that describe a repetition pattern of how often and when jobs should be run. The scheduler uses this to determine when to perform job executions.

How the Scheduler Works

The figure below describes the components of the scheduler and how the flow is when creating schedules, and resolving those schedules for processing of batch jobs in the engine.



A user creates a schedule, which is passed to the scheduler. The `SchedulerProducer` executes the `initiateProcessing()` method at 5-second intervals. This method retrieves active schedules from the system and constructs `JobCommand` messages for each schedule. These messages are then dispatched to the action channel in the scheduler service, which processes them based on priority. When using scheduled jobs, the queue priority corresponds to the priority value assigned during schedule creation or update.

The `SchedulerConsumer` contains a polling consumer (`engineConsumer`) component that consumes messages from the priority queue in the scheduler. This consumer receives commands and determines the appropriate action based on the command type, based on if it is a job start or job restart command. The responsibility for performing the actual execution is then delegated to the engine, which handles the job operations.

Scheduling Using Cron Expressions

Scheduling a job is done through the batch administration UI. Planning jobs on a given schedule is done by setting a CRON Expression on the `BatchScheduleInstance` column `REPETITION_PATTERN`. The cron expression should be defined in the Quartz format as follows:

Seconds are values between 0-59, minutes are values 0-59, hours are values between 0-23. Day of month can be values 1-31. Values for months can be 1-12 or JAN-DEC. Values for day of week can be 0-7 or MON-SAT. An asterisk (*) or a question mark (?) is defined as "blank". Furthermore, it is allowed to use a comma (,) to put in a list of values or a dash (-) to define a range.

Explanation of cron expression string.

Cron Expression	Description
`0 0 0 L * *`	Runs at midnight on the last day of every month.
`0 0 12 1 * *`	Runs at 12:00 PM (noon) on the 1st day of every month.
`0 0 0 ? * MON#1`	Runs at midnight on the first Monday of every month.
`0 */15 9-17 * * MON-FRI`	Runs every 15 minutes between 9 AM and 5 PM on weekdays (Mon–Fri).

For more information about CRON expressions, refer to the Spring documentation of `CronExpression`.

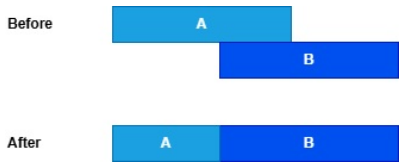
Resolving Schedules

The default implementation `BatchScheduleHelperImpl` sorts schedule instances by their creation date and accepts new instances that do not conflict with existing ones, i.e., those that do not overlap. When instances do overlap, the schedule with the most recent creation date takes precedence. This means that the older overlapping schedule will either be split if a newer schedule is inserted in the middle or partially overwritten if the newer schedule extends from the middle to or beyond the start or end. The graphs below illustrate two examples of how overlapping schedules are resolved, highlighting the precedence of newer schedules.

> **Important:** Instance resolution occurs only within the same schedule. If you create multiple schedules (for identical or different batch job types) the instances within those schedules remain isolated and do not interfere with each other.

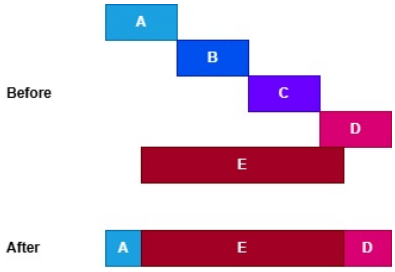
Example Resolution of Scheduler

Example of two overlapping schedules A and B. As B was created later and has precedence, it shortens the schedule for A.



Simple scheduling example.

Example of five schedules. E has precedence and overlaps with A, B, C, and D, causing A and D to be shortened and B and C to be overwritten entirely.



More complex scheduling example.

BatchSchedulerProducer

A scheduled service that continuously monitors and initiates due batch job schedules by running every 5 seconds to check for schedules that need execution. The service retrieves the current time and executes within the default batch context to call the batch scheduler service's schedule initiation process. It acts as the entry point for the batch scheduling system, automatically triggering the reservation and queuing of due schedules according to their configured schedules.

```
/**
 * Responsible for scheduling and initiating processing of batch jobs.
 */

public interface BatchSchedulerProducer {

    /**
     * Initiates the processing of scheduled batch jobs. This method is called periodically, to check and start jobs that are due for execution.
     */
    void initiateProcessing();

}
```

Config

Bean	Type	Purpose
`batchSchedulerProducer`	`BatchSchedulerProducer`	Bean with a scheudled method that initates all batch job schedules that are due for execution at a specified time.

BatchSchedulerConsumer

The engine consumer (`PollingConsumer` bean in `BatchSchedulerConsumerConfig`) acts as the central hub for command processing in the batch scheduler system. It serves as the bridge between the job command queue and the appropriate command handlers.

```
/**
 * Consumes job commands from the priority channel and routes them to the appropriate handler.
 * <p>
 * This component is responsible for:
 * 1. Polling the priority channel for job commands
 * 2. Identifying the appropriate command handler based on the command type
 * 3. Delegating command processing to the matching handler
 * 4. Ensuring only one command is processed at a time
 * <p>
 * The consumer works with a pluggable set of command handlers that each specialize
 * in a specific command type (manual start, restart, scheduled execution).
 */
@Bean
@SuppressWarnings("unchecked")
public PollingConsumer engineConsumer(
        PriorityChannel engineActionChannel,
        List<BatchSchedulingCommandHandler<?>> batchSchedulingCommandHandlers,
        PollSkipAdvice engineAtCapacityPollCondition
) {
    PollingConsumer pollingConsumer = new PollingConsumer(engineActionChannel, (message -> {
        Object payload = message.getPayload();
        if (!(payload instanceof JobCommand jobCommand)) {
            throw new BatchException(BatchErrorCodes.UNEXPECTED_COMMAND_TYPE_ERROR,
                    "Unsupported payload: " + payload.getClass());
        }

        final List<BatchSchedulingCommandHandler<JobCommand>> list = batchSchedulingCommandHandlers.stream()
                .filter(handler -> handler.isAssociated(jobCommand))
                .map(handler -> (BatchSchedulingCommandHandler<JobCommand>) handler)
                .toList();

        if (list.isEmpty()) {
            throw new BatchException(BatchErrorCodes.UNEXPECTED_COMMAND_TYPE_ERROR,
                    "Unsupported jobCommand: " + jobCommand.getClass());
        }

        if (list.size() > 1) {
            final String listOfHandlerNames = list.stream()
                    .map(c -> c.getClass().getSimpleName())
                    .collect(Collectors.joining(","));
            throw new BatchException(BatchErrorCodes.MULTIPLE_MATCHING_COMMAND_HANDLERS_ERROR,
                    "Matching handlers:" + listOfHandlerNames);
        }

        list.getFirst().handle(jobCommand);
```

```
    }));
    pollingConsumer.setMaxMessagesPerPoll(1);
    pollingConsumer.setAdviceChain(Collections.singletonList(engineAtCapacityPollCondition));
    return pollingConsumer;
}
```

The engine consumer is a key component in the batch scheduling system that:

- Continuously polls the `engineActionChannel` for new job commands
- Determines which command handler should process each command based on the `isAssociated()` method
- Ensures exactly one handler is matched for each command type
- Delegates the actual command processing to the appropriate handler
- Limits processing to one message per poll

This design separates concerns where:

1. The `BatchSchedulerProducer` generates commands and places them in the priority queue
2. The `engineConsumer` routes commands to the appropriate handlers
3. The command handlers (`JobManualStartCommandHandler` , `JobRestartCommandHandler` , and `JobScheduleStartCommandHandler`) implement the specific logic for each command type

Command handlers

BatchSchedulingCommandHandler Interface

```
/**
 * Defines a handler for processing specific types of batch job commands.
 *
 * @param <C> The specific type of JobCommand this handler processes
 */
public interface BatchSchedulingCommandHandler<C extends JobCommand> {

    /**
     * Processes the given job command.
     *
     * Implementations handle the command according to its type, typically by
     * interacting with the batch engine to execute the appropriate action.
     *
     * @param command The command to process (must not be null)
     */
    void handle(@NotNull C command);

    /**
     * Determines whether this handler can process the given job command.
     *
     * Used by the engine consumer to route commands to the appropriate handler.
     *
     * @param jobCommand The command to check (must not be null)
     * @return true if this handler can process the command, false otherwise
     */
    boolean isAssociated(@NotNull JobCommand jobCommand);
}
```

JobManualStartCommandHandler

```
/**
 * Handles manual job start commands by initiating immediate job execution.
 *
 * This command handler is responsible for processing manual job start requests
 * from the batch administration UI or API. It delegates to the batch engine to
 * start the job with the specified parameters.
 *
 * When a JobManualStartCommand is received by the engine consumer, this handler
 * is selected based on the isAssociated() method and its handle() method is called.
 */
public class JobManualStartCommandHandler implements BatchSchedulingCommandHandler<JobManualStartCommand> {


    /**
     * Processes a manual job start command by starting the specified job with parameters.
     *
     * @param command The command containing job name and parameters (must not be null)
     */
    @Override
    public void handle(final @NotNull JobManualStartCommand command) {
    }

    /**
     * Determines if this handler should process the given command.
     *
     * @param jobCommand The command to evaluate (must not be null)
     * @return true if the command is a JobManualStartCommand, false otherwise
     */
    @Override
    public boolean isAssociated(final @NotNull JobCommand jobCommand) {
    }
}
```

JobRestartCommandHandler

```
/**
```

```java
 * Handles job restart commands by restarting previously executed jobs.
 *
 * This command handler is responsible for processing job restart requests
 * from the batch administration UI or API. It delegates to the batch engine to
 * restart the job with the specified instance ID.
 *
 * When a JobRestartCommand is received by the engine consumer, this handler
 * is selected based on the isAssociated() method and its handle() method is called.
 */
public class JobRestartCommandHandler implements BatchSchedulingCommandHandler<JobRestartCommand> {

    /**
     * Processes a job restart command by restarting the job with the specified instance ID.
     *
     * @param command The command containing the job instance ID to restart (must not be null)
     */
    @Override
    public void handle(final @NotNull JobRestartCommand command) {
        // Implementation details...
    }

    /**
     * Determines if this handler should process the given command.
     *
     * @param jobCommand The command to evaluate (must not be null)
     * @return true if the command is a JobRestartCommand, false otherwise
     */
    @Override
    public boolean isAssociated(final @NotNull JobCommand jobCommand) {
        // Implementation details...
    }
}
```

JobScheduleStartCommandHandler

```java
/**
 * Processes scheduled job execution requests based on batch schedule definitions.
 *
 * This command handler is a key component of the batch scheduling system, responsible for:
 *
 * 1. Retrieving and validating schedule information
 * 2. Starting jobs according to their schedule
 * 3. Managing successful execution by calculating next run time
 * 4. Handling failures with exponential backoff retry logic
 * 5. Maintaining the schedule lifecycle (reservation, execution, release)
 *
 * When a JobScheduleStartCommand is received by the engine consumer, this handler
 * is selected based on the isAssociated() method and executes the job within the
 * appropriate batch context.
 */
@Component
@RequiredArgsConstructor
public class JobScheduleStartCommandHandler implements BatchSchedulingCommandHandler<JobScheduleStartCommand> {

    private static final long LATE_JOB_START_WARNING_LIMIT = 5;

    /**
     * Processes a scheduled job start command within a batch context.
     *
     * This method:
     * 1. Retrieves the schedule by its key
     * 2. Attempts to run the job based on schedule parameters
     * 3. Calculates the next run time based on success/failure
     * 4. Updates the schedule with the next run time
     *
     * The execution occurs within a batch context to ensure proper tenant context
     * and other environmental settings are maintained.
     *
     * @param command The command containing schedule information (must not be null)
     */
    @WithSpan
    @Override
    public void handle(final @NotNull JobScheduleStartCommand command) {
        // Implementation details...
    }

    /**
     * Determines if this handler should process the given command.
     *
     * @param jobCommand The command to evaluate (must not be null)
     * @return true if the command is a JobScheduleStartCommand, false otherwise
     */
    @Override
    public boolean isAssociated(final @NotNull JobCommand jobCommand) {
        // Implementation details...
    }
}
```

Scheduler Services

BatchSchedulerConfig

A Spring configuration class that sets up the following beans for the scheduler:

Bean	Type	Purpose
`batchJobControlMessageStore`	`JdbcChannelMessageStore`	Sets up a JDBC message store for persistent storage of batch job control messages with priority support, using a dedicated database table with configurable prefix
`engineActionChannel`	`PriorityChannel`	Creates a priority-based message channel backed by the JDBC message store for queuing batch job commands in order of priority

BatchSchedulerService

`BatchSchedulerService` is an interface for managing and executing batch job schedules, as well as adh-hoc jobs.

```
/**
 * Service responsible for managing and executing batch job schedules.
 * Provides functionality for initiating scheduled jobs based on time triggers
 * and running ad-hoc jobs with custom parameters.
 */
public interface BatchSchedulerService {

    /**
     * Initiates all batch job schedules that are due for execution at the specified time.
     *
     * @param currentTime The reference time used to determine which schedules are due for execution
     */
    void initiateDueSchedules(@NotNull final LocalDateTime currentTime);

    /**
     * Executes a specific job immediately with the provided parameters and priority.
     *
     * @param jobName       The name identifier of the job to execute
     * @param jobParameters The parameters to pass to the job execution
     * @param priority      The execution priority of the job (higher values indicate higher priority)
     */
    void initiateAdHocJobRun(@NotNull String jobName, @NotNull JobParameters jobParameters, @NotNull Integer priority);

    /**
     * Executes a specific schedule immediately with the provided parameters and priority.
     *
     * @param scheduleKey   The unique identifier of the schedule to execute
     * @param jobParameters Parameters to pass to the job execution
     * @param priority      The execution priority of the schedule (higher values indicate higher priority)
     */
    void initiateAdHocScheduleRun(@NotNull String scheduleKey, @NotNull JobParameters jobParameters,
                                  @NotNull Integer priority);

    /**
     * Restarts a failed or stopped job execution.
     *
     * @param jobInstanceId The unique identifier of the job instance to restart
     */
    void initiateRestartJob(@NotNull Long jobInstanceId);

    /**
     * Attempts to gracefully stop a running job execution.
     *
     * @param jobInstanceId The unique identifier of the job instance to stop
     * @return {@code true} if the stop signal was sent successfully, {@code false} otherwise
     * @throws NoSuchJobExecutionException     if the job execution does not exist
     * @throws JobExecutionNotRunningException if the job execution is already completed or stopped
     */
    boolean initiateStopJob(@NotNull Long jobInstanceId)
            throws NoSuchJobExecutionException, JobExecutionNotRunningException;

}
```

`initiateAdHocJobRun()` is called directly from `AdHocJobParametersFormFinalizer` only once to check and start the job immediately without linking it to any schedule instance using provided job parameters and priority. `initiateAdHocScheduleRun()` is called directly from `AdHocJobParametersFormFinalizer` only once to check and start the schedule immediately using provided job parameters and priority. It uses the current instance parameters of the schedule as defaults, which can be customized. Ad hoc runs do not require a repetition pattern and leaves the original schedule configuration unchanged.

BatchScheduleService

`BatchScheduleService` is responsible for lifecycle management of scheduling. It has methods for retrieving, reserving, and releasing a schedule.

```
/**
 * Service for lifecycle management of batch scheduling operations.
 * Provides functionality for creating, updating, retrieving, and managing batch schedules with support for reservation-based concurrent access control.
 */
public interface BatchScheduleService {

    /**
     * Attempts to reserve a schedule slot for the given date and time.
     *
     * @param dateTime the date and time for which to reserve a schedule slot
     * @return an optional containing the reservation key if successful, empty if no schedules available
     */
    @NotNull
    Optional<String> reserveBy(@NotNull LocalDateTime dateTime);
```

```java
    /**
     * Attempts to reserve a schedule slot for a schedule instance immediately.
     *
     * @param scheduleKey the key of the schedule that needs to be reserved
     * @return an optional containing the reservation key if successful, empty if no schedules available
     */
    @NotNull
    Optional<String> reserveBy(@NotNull String scheduleKey);

    /**
     * Releases a previously reserved schedule using the reservation key.
     *
     * @param schedule       the schedule type to release
     * @param reservationKey the key obtained from a previous reserve operation
     */
    void release(@NotNull BatchScheduleType schedule, @NotNull String reservationKey);

    /**
     * Retrieves batch schedules with optional sorting and pagination.
     *
     * @param sortByNextRun if true, sorts results by next run time
     * @param offset        starting position for pagination, null for no offset
     * @param pageSize      maximum number of results to return, null for no limit
     * @return list of batch schedule types matching the criteria
     */
    @NotNull
    List<BatchScheduleType> retrieve(LocalDateTime startDate, LocalDateTime endDate, boolean sortByNextRun,
                                     @Nullable Integer offset, @Nullable Integer pageSize);

    /**
     * Retrieves all schedules currently reserved with the given reservation key.
     *
     * @param reservationKey the reservation key to filter by
     * @return list of reserved batch schedule types
     */
    @NotNull
    List<BatchScheduleType> retrieveReserved(@NotNull String reservationKey);

    /**
     * Retrieves a specific schedule by its unique key.
     *
     * @param scheduleKey the unique identifier of the schedule
     * @return optional containing the schedule if found, empty otherwise
     */
    @NotNull
    Optional<BatchScheduleType> retrieveByKey(@NotNull String scheduleKey);

    /**
     * Retrieves multiple schedules by their unique keys.
     *
     * @param scheduleKeys list of schedule keys to retrieve
     * @return list of matching batch schedule types
     */
    @NotNull
    List<BatchScheduleType> retrieveByKeys(@NotNull List<String> scheduleKeys);

    /**
     * Retrieves the schedules associated with a specific batch job type.
     *
     * @param jobName the batch job type to search for
     * @return list of matching batch schedule types
     */
    @NotNull List<BatchScheduleType> retrieveByJobType(@NotNull BatchJob jobName);

    /**
     * Retrieves schedules associated with multiple batch job types.
     *
     * @param jobNames list of batch job types to search for
     * @return list of matching batch schedule types
     */
    @NotNull
    List<BatchScheduleType> retrieveByJobTypes(@NotNull List<BatchJob> jobNames);

    /**
     * Retrieves resolved schedule instances for a given schedule type.
     * Resolved instances are non-overlapping intervals where newer instances
     * take precedence over older overlapping ones.
     *
     * @param batchScheduleType the schedule type to resolve instances for
     * @return ordered list of non-overlapping schedule instances
     */
    @NotNull
    List<BatchScheduleInstance> retrieveResolvedInstances(@NotNull BatchScheduleType batchScheduleType);

    /**
     * Finds the active schedule instance for a specific date from resolved instances.
     *
     * @param resolvedInstances list of resolved schedule instances
     * @param dateTime          the date to find the active instance for
     * @return optional containing the active instance, empty if none found
     */
```

```
    @NotNull
    Optional<BatchScheduleInstance> retrieveResolvedInstance(@NotNull List<BatchScheduleInstance> resolvedInstances,
                                                             @NotNull LocalDate dateTime);

    /**
     * Creates a new schedule type for the specified batch job. It is created with an initial inactive schedule instance with start date as minDate and
end date as maxDate.
     *
     * @param batchJob the batch job to create a schedule for
     * @return the unique schedule key of the created schedule
     */
    @NotNull
    String create(@NotNull BatchJob batchJob);

    /**
     * Updates an existing schedule type, typically to modify the next run time.
     *
     * @param schedule the schedule to update
     * @throws BatchException if schedule not found
     */
    void update(@NotNull BatchScheduleType schedule);

    /**
     * Adds a new schedule instance to an existing schedule type and recalculates next run time.
     *
     * @param scheduleInstance the schedule instance configuration to add
     * @throws BatchException if schedule not found or description is blank
     */
    void add(@NotNull BatchScheduleInstanceCommand scheduleInstance);

    /**
     * Increments the retry attempt counter for a failed job execution and persists the change.
     *
     * @param batchScheduleType the schedule type to increment retry counter for
     */
    void incrementRetryCounter(@NotNull BatchScheduleType batchScheduleType);

    /**
     * Resets the retry attempt counter to zero for a successful job execution and persists the change.
     *
     * @param batchScheduleType the schedule type to reset retry counter for
     */
    void resetRetryCounter(@NotNull BatchScheduleType batchScheduleType);

    /**
     * Calculates the next scheduled run time based on resolved instances and previous execution.
     *
     * @param resolvedInstances non-overlapping ordered schedule instances
     * @param previousRunTime   the time of the previous execution
     * @return optional containing the next run time if calculable, empty otherwise
     */
    @NotNull
    Optional<LocalDateTime> calculateNextRun(@NotNull List<BatchScheduleInstance> resolvedInstances,
                                             @NotNull LocalDateTime previousRunTime);
}
```

BatchScheduleHelper

The interface `BatchScheduleHelper` contains a single method `retrieveResolvedInstancesInternal` and is used by the `BatchScheduleService`. See section Resolving schedules for a description of the default resolving of schedules which this class provides.

```
/**
 * Processes a collection of raw batch schedule instances to produce a list of resolved schedule instances.
 *
 * @param rawInstances a collection of {@link BatchScheduleInstance} objects to be processed. This collection
 *                     should contain instances with valid scheduling data.
 * @return a list of {@link BatchScheduleInstance} objects representing the resolved state of the input instances.
 */
List<BatchScheduleInstance> retrieveResolvedInstancesInternal(
        final @NotNull Collection<BatchScheduleInstance> rawInstances);
```

BatchScheduleJobRelationService

`BatchScheduleJobRelationService` is responsible for managing the relationship between the batch job and the schedule. This is where the logic for creating and deleting the relationship is done.

```
/**
 * Service for managing relationships between batch schedules and job instances.
 * Provides functionality to link schedules to job executions and retrieve relationship history.
 */
public interface BatchScheduleJobRelationService {

    /**
     * Creates a relationship (a {@link BatchScheduleJobRelationJpa}) between a schedule and a job instance and persists it.
     *
     * @param schedule      the schedule to link
     * @param jobInstanceId the job instance ID to link to the schedule
     * @throws BatchException if schedule not found
     */
    void linkScheduleToJobInstance(@NotNull BatchScheduleType schedule, @NotNull Long jobInstanceId);
```

```
    /**
     * Retrieves all job instance relationships for a given schedule.
     *
     * @param schedule the schedule to get relationships for
     * @return set of all job relations for the schedule
     */
    @NotNull
    Set<BatchScheduleJobRelation> getBatchScheduleJobInstanceRelation(@NotNull BatchScheduleType schedule);

    /**
     * Retrieves the most recent completed job instance linked to a schedule.
     *
     * @param schedule the schedule to get the latest completed instance for
     * @return optional containing the latest completed job relation, empty if none found
     */
    @NotNull
    Optional<BatchScheduleJobRelation> getLatestCompletedLinkedInstance(@NotNull BatchScheduleType schedule);

    /**
     * Retrieves the most recent completed job instances for multiple schedules.
     *
     * @param schedules list of schedules to get latest completed instances for
     * @return list of latest completed job relations for each schedule
     */
    @NotNull
    List<BatchScheduleJobRelation> getLatestCompletedLinkedInstances(@NotNull List<BatchScheduleType> schedules);

    /**
     * Retrieves the most recent job instance (any status) linked to a schedule.
     *
     * @param schedule the schedule to get the latest instance for
     * @return optional containing the latest job relation, empty if none found
     */
    @NotNull
    Optional<BatchScheduleJobRelation> getLatestLinkedInstance(@NotNull BatchScheduleType schedule);

    /**
     * Retrieves the most recent job instances (any status) for multiple schedules.
     *
     * @param schedules list of schedules to get latest instances for
     * @return list of latest job relations for each schedule
     */
    @NotNull
    List<BatchScheduleJobRelation> getLatestLinkedInstances(@NotNull List<BatchScheduleType> schedules);
}
```

BatchScheduleRepository

`BatchScheduleRepository` is responsible for the database operations for the scheduling. This is where the logic for creating, updating, and deleting schedules is done.

```
/**
 * Repository for batch schedule data access operations.
 * Provides persistence, retrieval, and reservation management for batch schedules.
 */
public interface BatchScheduleRepository {

    /**
     * Reserves schedules due for execution and assigns them a UUID reservation key.
     *
     * @param dateTime the time to reserve schedules for
     * @return optional containing reservation key if schedules were reserved, empty otherwise
     */
    @NotNull
    Optional<String> reserveBy(@NotNull LocalDateTime dateTime);

    /**
     * Reserves a single schedule and assigns it a UUID reservation key.
     *
     * @param scheduleKey the key of schedule that need to be reserved
     * @return optional containing reservation key if schedules were reserved, empty otherwise
     */
    @NotNull
    Optional<String> reserveBy(@NotNull String scheduleKey);

    /**
     * Releases a specific schedule reservation.
     *
     * @param scheduleKey    the schedule to release
     * @param reservationKey the reservation key to validate
     * @throws BatchException if reservation not found
     */
    void release(@NotNull String scheduleKey, @NotNull String reservationKey);

    /**
     * Persists a schedule type entity.
     */
    void persist(@NotNull AbstractBatchScheduleTypeJpa scheduleType);

    /**
     * Persists a schedule instance entity.
     */
```

```
    void persist(@NotNull AbstractBatchScheduleInstanceJpa scheduleInstance);

    /**
     * Retrieves schedules with optional sorting and pagination, ordered by next run time and creation date.
     *
     * @param sortByNextRun if true, sorts by next run time ascending
     * @param offset        starting position for pagination, null for no offset
     * @param pageSize      maximum results to return, null for no limit
     * @return list of schedule types matching criteria
     */
    @NotNull
    List<? extends AbstractBatchScheduleTypeJpa> retrieve(LocalDateTime startDate, LocalDateTime endDate,
                                                          boolean sortByNextRun, @Nullable Integer offset,
                                                          @Nullable Integer pageSize);

    /**
     * Retrieves all schedules reserved with the given key.
     */
    @NotNull
    List<? extends AbstractBatchScheduleTypeJpa> retrieveReserved(@NotNull String reservationKey);

    /**
     * Retrieves schedules associated with specific job types.
     */
    @NotNull
    List<? extends AbstractBatchScheduleTypeJpa> retrieveJobTypes(@NotNull List<BatchJob> jobNames);

    /**
     * Retrieves a single schedule by its unique key.
     */
    @NotNull
    Optional<? extends AbstractBatchScheduleTypeJpa> getBatchScheduleType(@NotNull String scheduleKey);

    /**
     * Retrieves multiple schedules by their keys.
     */
    @NotNull
    List<? extends AbstractBatchScheduleTypeJpa> getBatchScheduleTypes(@NotNull List<String> scheduleKeys);

    /**
     * Retrieves all job instance relationships for a schedule.
     */
    @NotNull
    Set<? extends AbstractBatchScheduleJobRelationJpa> getBatchScheduleJobInstanceRelation(
            @NotNull BatchScheduleType schedule);

    /**
     * Retrieves all schedule instances for a given schedule type.
     */
    @NotNull
    List<? extends AbstractBatchScheduleInstanceJpa> retrieveInstances(@NotNull BatchScheduleType batchScheduleType);

    /**
     * Retrieves the most recent job relations for multiple schedules using subquery for max creation time.
     */
    @NotNull
    List<? extends AbstractBatchScheduleJobRelationJpa> getLatestLinked(@NotNull List<BatchScheduleType> schedules);

    /**
     * Retrieves the most recent completed job relations using native SQL with job execution status filtering.
     */
    @NotNull
    List<? extends AbstractBatchScheduleJobRelationJpa> getLatestCompletedLinkedInstances(
            @NotNull List<BatchScheduleType> schedules);

    /**
     * Retrieves the first completed or abandoned job relation for a job type, ordered by creation time.
     * Used to determine default start time for job execution queries.
     *
     * @param jobType the job type to search for
     * @return optional containing the first completed/abandoned job relation, empty if none found
     */
    @NotNull
    Optional<? extends AbstractBatchScheduleJobRelationJpa> getJobRelationOfFirstCompletedOrAbandonedJobExecutionByJobType(
            @NotNull String jobType);
}
```

BatchScheduleFactory

`BatchScheduleFactory` is responsible for the creation of BatchSchedule related JPAs and other related instances. Supports creating different JPAs depending on if multitenancy is enabled or not.

> This interface should always be used to initialize the JPA objects and other classes.

```
/**
 * Factory interface for creating batch schedule related entities and proxies.
 */
public interface BatchScheduleFactory {

    /**
     * Creates a new batch schedule instance based on the provided command.
```

```
     *
     * @param command the command object containing the parameters for the batch schedule instance creation
     * @return a new {@link AbstractBatchScheduleInstanceJpa} representing the created schedule instance
     */
    AbstractBatchScheduleInstanceJpa createBatchScheduleInstance(BatchScheduleInstanceCommand command);

    /**
     * Creates a new batch schedule type for a specific job.
     *
     * @param scheduleKey the unique key identifying the schedule type
     * @param batchJob the {@link BatchJob} associated with this schedule type
     * @return a new {@link AbstractBatchScheduleTypeJpa} representing the schedule type
     */
    AbstractBatchScheduleTypeJpa createBatchScheduleType(String scheduleKey, BatchJob batchJob);

    /**
     * Creates a proxy for a batch schedule instance for the specified date range.
     *
     * @param startDate the start date of the proxy instance
     * @param endDate the end date of the proxy instance
     * @param batchScheduleInstance the original {@link BatchScheduleInstance} to proxy
     * @return a proxied {@link BatchScheduleInstance} limited to the provided dates
     */
    BatchScheduleInstance createBatchScheduleInstanceProxy(LocalDate startDate, LocalDate endDate,
                                                           BatchScheduleInstance batchScheduleInstance);

    /**
     * Creates a relation entity between a batch schedule type and a job instance.
     *
     * @param batchScheduleType the {@link AbstractBatchScheduleTypeJpa} to relate
     * @param jobInstanceId the ID of the job instance to associate with the schedule type
     * @return a new {@link AbstractBatchScheduleJobRelationJpa} representing the relation
     */
    AbstractBatchScheduleJobRelationJpa createBatchScheduleJobRelation(AbstractBatchScheduleTypeJpa batchScheduleType,
                                                                       Long jobInstanceId);

}
```

TenantAwareJobParameterProvider

`TenantAwareJobParameterProvider` is responsible for getting job parameters with tenant information for multi-tenant batch job execution scenarios.

```
/**
 * Responsible for getting job parameters with tenant information for multi-tenant batch job execution scenarios.
 */
public interface TenantAwareJobParameterProvider {

    /**
     * Return job parameters for given instance and job parameters.
     * Add tenant id if the batch job has not been scheduled to run as a certain tenant and if there is a tenant id on the schedule.
     * Also validate whether tenant is allowed to run the job.
     *
     * @param currentInstance current instance being processed.
     * @param commandMessage a {@link JobScheduleStartCommand} object containing name of job, a schedule key and a reservation key.
     *                       It also contains job parameters for ad hoc schedule start
     * @return a {@link JobParameters} of the job after adding the tenant id if multi tenancy is enabled.
     */
    JobParameters getJobParameters(JobScheduleStartCommand commandMessage, BatchScheduleInstance currentInstance);

    /**
     * Return job parameters with current tenant id for ad hoc instance start.
     * Add tenant id if the batch job has not been assigned a tenant id as input parameters and multi tenancy is enabled for batch job.
     * Also validate whether tenant is allowed to run the job.
     *
     * @param props Properties instance of job parameters of the instance.
     * @return a {@link JobParameters} of the job after adding the tenant id if multi tenancy is enabled.
     */
    JobParameters getJobParameters(Properties props);
}
```

TenantAwareJobParameterProviderHelper

`TenantAwareJobParameterProviderHelper` is a helper interface for retrieving tenant information and validating tenant access for batch job execution in multi-tenant environments.

```
/**
 * Helper interface for managing tenant-aware batch job parameter operations.
 * <p>
 * This interface provides methods for retrieving tenant information and validating
 * tenant access for batch job execution in multi-tenant environments. Implementations
 * can customize tenant handling logic based on specific project requirements.
 * </p>
 */
public interface TenantAwareJobParameterProviderHelper {

    /**
     * Retrieves the tenant ID to use for ad-hoc batch job runs.
     *
     * @return the tenant ID string to use for the batch job execution, or {@code null}
     *         if no tenant context is available or multi-tenancy is disabled
     */
    String getTenantIdForAdhocBatchJobRun();
```

```
    /**
     * Validates whether the current execution context has access to run batch jobs
     * for the specified tenant.
     *
     * @param requestedTenantId the tenant ID for which batch job execution is requested.
     *                          Must not be {@code null} or empty when multi-tenancy is enabled
     */
    void validateTenantAccess(String requestedTenantId);
}
```

Batch Job Reporting

Foundation Batch includes reporting capabilities that automatically capture execution details, errors, and custom metrics during job processing. Reports are generated on-demand when a user tries to download a report from the administration UI.

Key Features:

- Automatic collection of execution metadata and item statistics
- Support for custom reporting details and business metrics
- Aggregation of numeric values with configurable formatting

To enable batch job reporting:

1. **Add dependency**: Include `batch-reporting-service` in your Gradle dependencies
2. **Import configuration**: Add `BatchReportingConfig` to your application configuration imports
3. **Use collector**: Inject `BatchReportingDetailCollector` into your batch components to add custom reporting details. Reports are automatically generated and accessible through the batch administration interface after job completion.

For detailed implementation guidance, see the DD130 - Batch Job Reporting documentation.

Multitenancy

This section describes the default features supporting multitenant applications. For comprehensive information about multitenancy, please refer to DD130 - Multitenancy.

Core Features

Preventing data leaks when running batch jobs in a multitenant environment is critical. Foundation supports two methods for scheduling a job to run under a specific tenant context:

1. **Default behavior**: If no job parameters are provided, the tenant ID from the `batch_schedule_instance` is used
2. **Parameter override**: If a `TENANT_ID` job parameter is provided, it takes precedence over the default

See the configuration details below for implementation guidance.

Scheduling Jobs for Specific Tenant Contexts

You can configure job parameters to display a tenant dropdown list in the edit schedule modal. The selected tenant ID determines the context under which the batch job executes.

Implementation Example: The Foundation reference app demonstrates this in the `TenantAwareBatchjob` implementation, which includes:

- `TenantAwareBatchJobExampleJobParameters` - extends `TenantAwareJobParameters`
- `BatchJobTenantServiceImpl` - handles tenant-specific logic

By extending `TenantAwareJobParameters`, a job parameter with the key `TENANT_ID` is automatically added. Note the `getParameterOrder` method override, which displays the tenant list only when the current user belongs to the global tenant:

> **Note:** If the `TENANT_ID` job parameter is not provided, the batch job will run under the tenant that created the schedule.

```java
@Override
public List<String> getParameterOrder() {
    if (tenantContextHelper.isGlobalTenant()) {
        return List.of(MultiTenancyConstants.TENANT_ID);
    }
    return List.of();
}
```

Partitioning Steps Across Multiple Tenants

Foundation supports the possibility to run batch jobs under a global tenant (with access to all data) while partitioning batch items by their owning tenants. This ensures subsequent steps execute in the correct tenant context, preventing data leaks.

Implementation Process:

1. **Schedule the job** to run as the global tenant
2. **First step**: The reader provides relevant items with their respective tenant information

- No business logic should be implemented in this step (runs as global tenant)
- `BatchItem` must implement `TenantAwareBatchItemValue`

3. **Subsequent steps**: Use `BatchItemTenantPartitioner` to partition batch items by tenant

- Reader/processor/writer components now run in the correct tenant context
- Business logic should be implemented in these steps to ensure data isolation

Reference: See the `TenantAwareBatchjob` implementation in the Foundation reference application for a complete example.

Restricting Batch Job Access by Tenant

You can limit which batch jobs specific tenants can schedule by implementing the `BatchJobTenantService#isAccessibleByCurrentTenant(BatchJob batchJob)` method.

> **Implementation Required:** Projects must implement this service themselves. Refer to the Foundation reference app for an example implementation.

Tenant-Filtered Entities

To prevent data leakage, Foundation provides multitenant versions of the following JPA entities:

- `MultitenantBatchItemJpa`
- `MultitenantBatchScheduleInstanceJpa`
- `MultitenantBatchScheduleTypeJpa`
- `MultitenantBatchScheduleJobRelationJpa`

Factory Pattern Implementation

To ensure proper JPA entity usage, Foundation implements factory classes:

- `BatchItemFactory`
- `BatchScheduleInstanceFactory`

Best Practice: Always use these factories when creating batch JPA entities to ensure the correct class implementation for your multitenant environment.

Spring Batch Integration Considerations

Spring Batch's native database tables present challenges for tenant filtering. Foundation addresses this with the `BatchJobInstanceTenantRelation` JPA entity.

Important: When querying job instances, always include a tenant-filtered table in the query join to ensure proper tenant isolation.

Components

The following standard components come with the batch-multitenancy module.

Component	Description	Reference Section
BatchItemTenantPartitioner	Partitioner partitions the batch items using the unique tenant ids on the batch items for the step, each unique tenant id is put on the StepExecutionContext.	BatchItemTenantPartitioner
TenantAwareBatchItemValue	Interface that should be applied to classes extending batch item if they should also be tenant aware. The tenant id put on the batch item will be the tenant id persisted on the batch item in the database	
TenantAwareJobParameters	Abstract class that can be used together with the `BatchJobTenantService` to provide a selectbox with tenants, the chosen tenant will be the tenant id set on the BatchSchedule and therefore also the tenant which the batchjob will run as.	

API

The following section describes the services and REST API of the batch application.

Services

The services are described throughout the module sections above. Refer to the tables and reference sections to read about a specific one.

Engine Services

Service	Description	Reference Section
`BatchEngine`	Main service for managing batch jobs - handles starting, restarting, stopping and recovering stuck batch jobs	BatchEngine
`BatchEngineThrottleService`	Service that determines if the batch engine has capacity to accept additional jobs based on system resource availability	BatchEngineThrottleService

Core Services

Service	Description	Reference Section
`BatchJobRegistryService`	Service for providing controlled access to batch jobs	BatchRegistryService
`BatchJobTenantService`	Service for managing tenant access and visibility for batch jobs, only needs to be implemented in a multitenant system	BatchJobTenantService
`BatchJobOperatorService`	High-level operations for controlling Spring Batch jobs, including graceful job stopping	BatchJobOperatorService
`BatchJobConfigurationService`	Service for fetching batch job configuration parameters from system parameters	BatchJobConfigurationService
`ParameterConverter`	Interface for converting user inputs into JobParameters for batch jobs	ParameterConverter

Item Services

Service	Description	Reference Section
`BatchItemService`	Service for managing batch items - adding, reading, updating statuses, deletion and recovery	BatchItemService
`BatchItemFactory`	Factory for creating batch item related entities.	BatchItemFactory
`BatchItemRepository`	Data access component for batch item operations - retrieval, lookup, deletion and queries	BatchItemRepository
`BatchItemJobBuilderFactory`	Factory for creating batch jobs using `DirectedPathBatchJobBuilder`	Job and Step Factory and Builders
`BatchItemStepBuilderFactory`	Factory for creating various types of batch processing steps	Job and Step Factory and Builders
`BatchItemReader`	Reader interface for batch items extending Spring Batch's ItemReader	BatchItemReader
`BatchItemProcessor`	Processor for transforming batch items from input to output type	BatchItemProcessor
`BatchItemWriter`	Writer interface for batch item processing with support for side effects and follow-up creation	BatchItemWriter
`AggregateItemService`	Service for managing aggregate items in batch processing workflows	AggregateItemService
`AggregateItemMerger`	Interface for merging and aggregating batch items through Input → Aggregate → Output pipeline	AggregateItemMerger

Scheduler Services

Service	Description	Reference Section
`BatchScheduleService`	Service for lifecycle management of scheduling - retrieving, reserving and releasing schedules	BatchScheduleService
`BatchSchedulerService`	Service responsible for managing and executing batch job schedules.	BatchSchedulerService
`BatchJobScheduler`	Service for scheduling and executing batch jobs based on database schedules	BatchJobScheduler
`BatchScheduleJobRelationService`	Service for managing relationships between batch jobs and schedules	BatchScheduleJobRelationService
`BatchScheduleRepository`	Repository for database operations related to scheduling	BatchScheduleRepository
`BatchScheduleHelper`	Interface for resolving schedule conflicts and processing schedule instances	BatchScheduleHelper
`TenantAwareJobParameterProvider`	Interface for for getting job parameters with tenant information for multi-tenant batch job execution scenarios.	TenantAwareJobParameterProvider
`TenantAwareJobParameterProviderHelper`	Interface for managing tenant-aware batch job parameter operations.	TenantAwareJobParameterProviderHelper
`BatchScheduleFactory`	Interface for handling creation of BatchSchedule related instances.	BatchScheduleFactory

Batch Administration

Service

Module `administration-batch-service` contains classes for the dynamic forms used for scheduling which are used in the batch overview page. These are used in the REST module.

REST

Module `administration-batch-rest`. Apart from the REST APIs and their services described below, this module also contains table-related classes for the tables shown on the batch overview page.

BatchRepositoryRestController

This API provides repository operations related to batch processing. URL prefix: `/rest/api/admin/batch/repository`.

HTTP Method	Endpoint	Description	Parameters	Response Type	Security Role Required
GET	`/jobs`	Retrieves all job definitions	None	`JobDefinitionDto[]`	`BATCH_READ`
POST	`/job/stepNames`	Retrieves step names for specified jobs	`JobNamesDto`	`StepNameDto[]`	`BATCH_READ`
POST	`/instances/query`	Retrieves all job instance in a format suitable to a selectbox	`DefaultQuerySelectRequest`	`DefaultQuerySelectOption[]`	`BATCH_READ`

BatchInstanceRestController

This API provides operations for queueing batch engine actions. URL prefix: `/rest/api/admin/batch/instance`.

HTTP Method	Endpoint	Description	Parameters	Response Type	Security Role Required
POST	`/restart`	Queues a restart command for a job instance	`jobInstanceId`	`Void`	`BATCH_SCHEDULER_WRITE`
POST	`/stop`	Initiates stop of a running job instance	`jobInstanceId`	`JobExecutionDto`	`BATCH_SCHEDULER_WRITE`
GET	`/adhoc/jobs/{jobName}/parameters`	Gets form instance for an ad-hoc job start	`jobName`, `scheduleKey` (optional)	`FormResource<FormInstance>`	`BATCH_SCHEDULER_READ`
POST	`/adhoc`	Starts an ad-hoc job execution with parameters	`FormInstance` (request body)	`FormResultResource<String>`	`BATCH_SCHEDULER_WRITE`

BatchScheduleRestController

This API provides operations for managing batch schedules. URL prefix: `/rest/api/admin/batch/schedule`.

HTTP Method	Endpoint	Description	Parameters	Response Type	Security Role Required
GET	`/`	Retrieves all schedules	`jobName` (optional), `withOutNext` (optional)	`BatchScheduleTypeDto[]`	`BATCH_SCHEDULER_READ`
POST	`/`	Creates a new schedule	`BatchScheduleTypeRestCommand` (request body)	`BatchScheduleTypeDto`	`BATCH_SCHEDULER_WRITE`
GET	`/{scheduleKey}`	Retrieves a specific schedule	`scheduleKey`	`BatchScheduleTypeDto`	`BATCH_SCHEDULER_READ`
DELETE	`/{scheduleKey}`	Deletes a specific schedule	`scheduleKey`	`BatchScheduleTypeDto`	`BATCH_SCHEDULER_WRITE`
GET	`/{scheduleKey}/runs`	Retrieves planned runs for a schedule	`scheduleKey`, `numberOfRuns`	`PlannedRunDto[]`	`BATCH_SCHEDULER_READ`

PUT	`/{scheduleKey}/release`	Releases a schedule	`scheduleKey`	`BatchScheduleTypeDto`	`BATCH_SCHEDULER_WRITE`
GET	`/scheduled/jobs/{jobName}/parameters/{scheduleKey}`	Gets form instance for scheduled job parameters	`jobName`, `scheduleKey`	`FormResource<FormInstance>`	`BATCH_SCHEDULER_READ`
POST	`/scheduled`	Updates schedule with form parameters	`FormInstance` (request body)	`FormResultResource<String>`	`BATCH_SCHEDULER_WRITE`

Batch Item Administration

REST

Module `administration-batch-item-rest`. Apart from the REST API described below, this module also contains table-related classes for the tables shown on the batch item page.

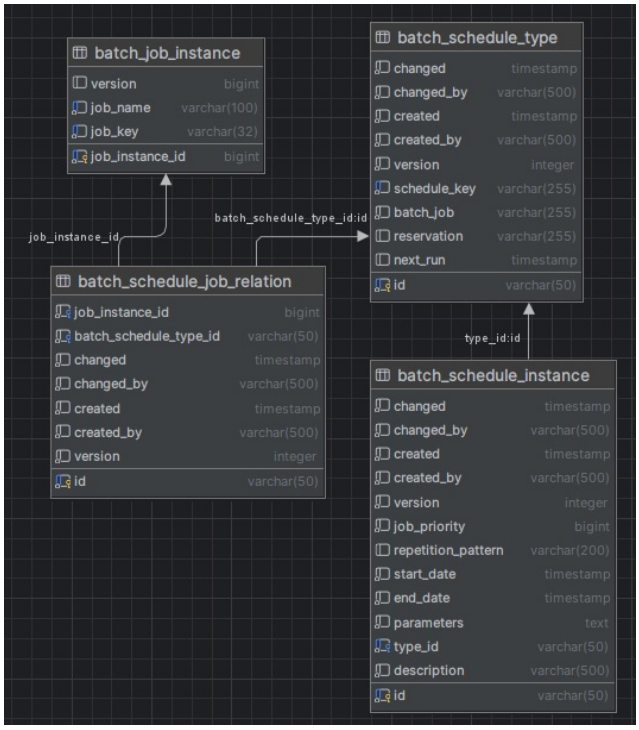
BatchItemRestController

This API provides operations for managing batch items. URL prefix. `/rest/api/admin/batch/item`

HTTP Method	Endpoint	Description	Parameters	Response Type	Security Role Required
GET	`/status`	Retrieves batch item status counts for a job instance	`jobInstanceId`	`BatchItemStatusCountDto[]`	`BATCH_READ`
GET	`/status-to-update`	Retrieves available status options for updates	None	`BatchItemStatusDto[]`	`BATCH_READ`
POST	`/recover/processing`	Resets processing of failed items	`jobInstanceId`, `interStepReference`	`BatchItemRecoverResultDto`	`BATCH_WRITE`
PUT	`/status`	Updates the status of a batch item	`batchItemId`, `status`	`BatchItemStatusUpdateDto`	`BATCH_WRITE`
DELETE	`/bulk`	Deletes items in bulk	`request`	`Item`	`BATCH_WRITE`
DELETE	`/`	Deletes specified batch items	`batchItemsDeleteDto`	`Item`	`BATCH_WRITE`

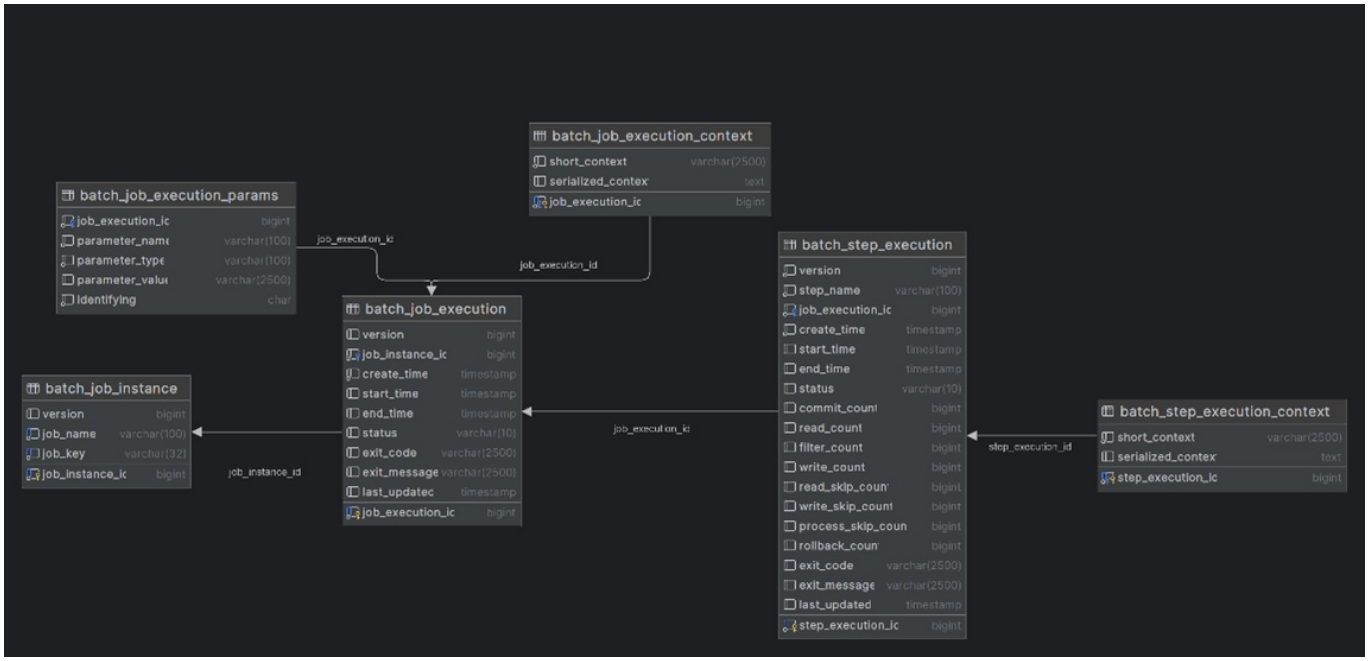
Data Model

Schedule Data Model



The data model for scheduling is very simple. There are also 3 tables related to scheduling: `batch_schedule_type`, `batch_schedule_instance`, and `batch_schedule_job_relation`. The `batch_schedule_type` has the role of pessimistic locking for scheduling and managing when the next run will be triggered. The `batch_schedule_job_relation` is just a relationship table between the batch job instance and the schedule. The `batch_schedule_instance` table contains the information regarding the scheduling of batch jobs. Some important context is no matter what schedules are made, nothing is deleted from the database, so all the `batch_schedule_instances` will persist, and the actual final scheduling is calculated from the `batch_schedule_instances` table.

Batch Data Model



The diagram in the image above shows how everything batch-related is stored in the database.

Table Name	Description	Example
Batch_job_instance	Stores a reference to the batch job. Most of the context relevant for the batch job instance is in the Java code.	An instance could be a job that processes payroll for employees each month.
Batch_job_execution	Each execution of a job has a unique entry in the database. This represents an execution of the instance.	For the payroll processing instance, there would be a new execution entry for each monthly run, such as January, February, etc.
Batch_job_execution_params	Contains information specific to the job execution. Two executions of the same instance can have different parameters.	Parameters could include the date range for payroll calculation, such as "start_date" and "end_date" for a specific month.
Batch_job_execution_context	Used to recreate the state if the job crashes hard. Not recommended due to potential artifacts from previous runs.	Stores the current record ID being processed in a payroll file, so the job can resume from that record on restart.
Batch_step_execution_context	Similar to Batch_job_execution_context, but for steps.	Stores context like the last processed employee ID in a step that calculates taxes for payroll.
Batch_step_execution	Records each execution of a step. Useful for managing jobs and debugging errors.	If the step "Calculate taxes" within the payroll job fails, a new step execution is recorded for each retry attempt.

Note that the implementations for the batch jobs are only defined in the Java code. This means the configuration of the batch job is done with Spring beans. If you have problems with configuration of batch jobs, look at the Getting Started with Batch jobs or the FAQ section.

Batch Item Data Model

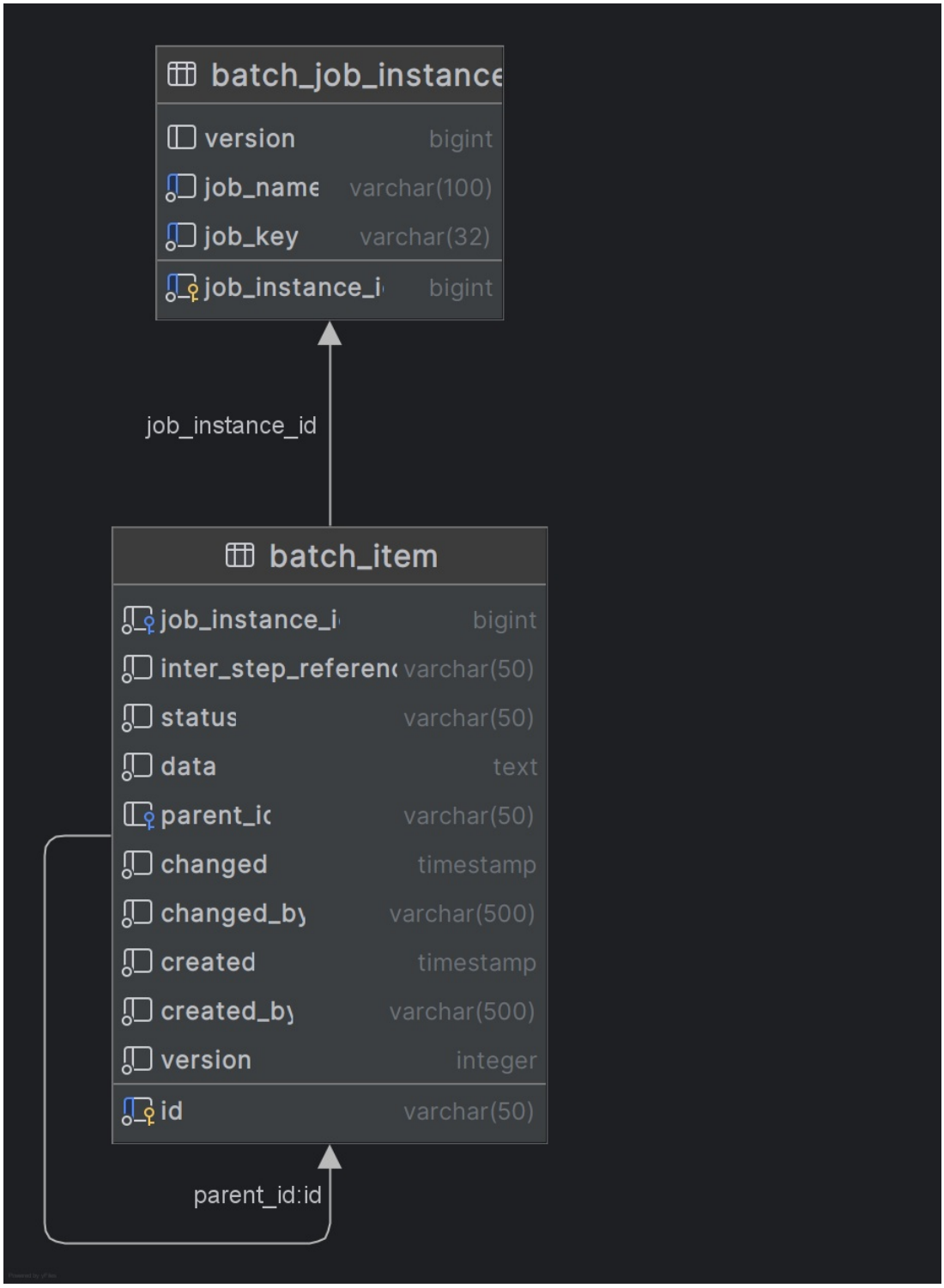


Table Name	Description	Example
Batch_item	Stores a batch item with its value (data).	A batch item could contain data about a citizen, for processing its payroll.

Configurations and Service Extensions

This chapter describes all the relevant practical information for potential users of the batch component to start using it.

Code Integration

This section contains a guide on how to implement Foundation Batch in your project.

Gradle Dependencies

> **Mandatory**

To use Foundation Batch, you need to add some API Gradle dependencies based on which modules you need. The batch component contains the following Gradle modules:

Module	Description	Additional Dependencies
`batch-core-service`	Holds error handling, system parameter configuration and classes representing batch jobs and steps.	-
`batch-engine-service`	ParameterConverter, abstract builders for jobs and steps, main service for job management.	`batch-core-service`
`batch-item-service`	Services for batch items and aggregate items.	`batch-core-service`

`batch-item-components`	All item abstractions and step components (reader, processor, writer, listener). Builders for jobs and steps. Partitioning.	`batch-core-service`, `batch-item-service`
`batch-scheduler-service`	Scheduler services.	`batch-core-service`
`batch-scheduler-producer`	Scheduler producer, for creating job commands based on schedules.	`batch-core-service`, `batch-scheduler-service`
`batch-scheduler-consumer`	Scheduler consumer, for initiating processing of batch jobs based on job commands.	`batch-scheduler-service`, `batch-engine-service`
`batch-multitenancy-service`	Service for batch multitenancy.	`batch-item-service`, batch-engine-service , batch-item-components`
`administration-batch-rest`	REST for the administration batch overview page.	`batch-scheduler-service`, `administration-batch-service`
`administration-batch-service`	Service for the administration batch overview page.	`batch-scheduler-service`
`administration-batch-item-rest`	REST for the administration batch item page.	`batch-item-service`

Application Configuration Imports

> **Mandatory**

Add the appropriate imports to your application API config. In the example below, the modules are imported into a separate BatchConfig that can be imported to the application API config.

```
@Configuration
@Import({
        BatchCoreConfig.class,
        BatchEngineConfig.class,

        BatchSchedulerConfig.class,
        BatchSchedulerManagerConfig.class,
        BatchSchedulerProcessorConfig.class,

        BatchItemConfig.class,
        BatchItemComponentsConfig.class,

        BatchMultitenancyConfig.class,

        AdminBatchRestConfig.class,
        AdminBatchServiceConfig.class,
        AdminBatchItemRestConfig.class,

        //batch jobsMigrati
})

public class BatchConfig {

}
```

```
@Import({
        //...
        BatchConfig.class,
})
@Configuration
@ComponentScan(
        //...
)

public class BusinessApiConfig {
    //...
}
```

The modules can be imported separately as required. For example, it can be split into two applications: the administration app with minimal config, and a batch worker which includes the scheduler and engine.

```
@Configuration
@Import({
        BatchCoreConfig.class,
        BatchSchedulerConfig.class,
        BatchItemConfig.class,

        AdminBatchRestConfig.class,
        AdminBatchServiceConfig.class,
        AdminBatchItemRestConfig.class,
})

public class BatchAdminConfig {

}
```

```
@Configuration
@Import({
```

```
        BatchCoreConfig.class,
        BatchEngineConfig.class,

        BatchSchedulerConfig.class,
        BatchSchedulerConsumerConfig.class,
        BatchSchedulerProducerConfig.class,

        BatchItemConfig.class,
        BatchItemComponentsConfig.class,

        // batch jobs
})

public class BatchWorkerConfig {

}
```

Configuration Files for Batch Jobs

A batch job configuration class is created with the `@Configuration` annotation from Spring. This is where you define the structure of your batch job along with beans of components that are required during execution.

> **Note:** [Getting started with Batch jobs](/C0200-User-Guides/Getting-started-with-batch-jobs) provides a quick overview for which parts are required when creating a new batch job. Examples of various architectural use cases are provided as well.

Take a look at the code below for the basic structure of a batch job configuration file.

```
/**
 * Configuration class for a specific batch job.
 */
@Configuration
@RequiredArgsConstructor
public class SequentialBatchJobExampleConfig {

    // dependencies

    // Job config

    // Step config(s)

    // Reader, processor, writer bean definitions

}
```

Then, an additional configuration file is set up for component scanning a specific package hierarchy where the batch job configuration file is located. These files can import multiple batch job configuration classes. This configuration should then be imported into the application where the batch job should be located.

```
/**
 * Configuration class for component scanning the package with the batch job(s).
 */
@Configuration
@ComponentScan(
        basePackages = {"nc.foundation.reference.example.batch"},
        excludeFilters = {@ComponentScan.Filter(type = FilterType.ANNOTATION, value = Configuration.class)}
)
@Import({
        SequentialBatchJobExampleConfig.class,
        PartitionsBatchJobExampleConfig.class,
        MultipleStepsBatchJobExampleConfig.class,
})
public class BatchJobExamplesConfig {

}
```

Splitting Batch Job Configurations

When using a setup where the administration application contains only a minimal batch configuration, and a separate worker application is responsible for handling schedules and executing jobs in the engine, it is beneficial to also separate the batch job definitions. This prevents the need to import the entire batch configuration into the administration application.

The administration modules only require access to the following classes for each batch job:

- `<job_name>JobConstants`
- `<job_name>JobParameters`
- `<job_name>ParametersValidator`

Therefore, ensure that these classes are placed in their own dedicated Spring configuration file. This configuration should be imported into both the administration application and the worker application.

The worker application should then contain the remaining batch job configuration, including all related components and services.

Configuration of System Parameters

> **Mandatory**

In this section, we describe the required system parameter attributes of a batch job.

Attribute	Description
timeout	Specifies how long a batch job needs to run before it can be recovered and its job status can be changed. E.g., if timeout is set to 10 (minutes), the duration between the job start and current time needs to be at least 10 minutes.

allowParallelJobExecution	Specifies whether a job is allowed to have multiple instances running in parallel. If it is not stated, the default value will be false meaning the job is never running in parallel.

Other Configurable Settings

Since we utilize Spring Boot's BatchAutoConfiguration, certain batch properties can be overridden to customize behavior if needed. All configurations default to Spring Batch's auto-configuration settings, except for the properties specified below.

To override any of the settings below, or other's from Spring Batch, specify the property in the `project-system.properties` file like below, replacing `{property_name}` with the desired property and `{value}` with your configuration. The Foundation defaults are specified in `foundation-system.properties`.

Property	Foundation Default	Description
`spring.batch.jdbc.isolation-level-for-create`	`READ_COMMITTED`	Specifies the transaction isolation level used when creating Spring Batch metadata tables and job execution records. > This setting should match your database's isolation level. Foundation defaults to `READ_COMMITTED` to align with PostgreSQL's default isolation level.
`spring.batch.job.enabled`	`false`	Controls whether Spring Batch jobs are automatically started on application startup. When set to `true`, you must also specify `spring.batch.job.name` to indicate which job should be executed. If multiple jobs are defined and no job name is specified, Spring Boot will fail to start with an error. > Override this setting only when you need to execute a specific batch job automatically on application startup for one-time execution scenarios. For normal batch worker operations, keep this as `false` to prevent unintended automatic job execution.

Properties

The batch modules allow for changing some properties using Spring property files. To customize these properties, add the following entries to your `datasource-fd01.properties` file:

BatchCoreProperties

Configuration prefix: `nc.foundation.libraries.batch.core.base`

Property	Default Value	Description
`messageTablePrefix`	`batch_`	Prefix for batch message tables

Example:

```
nc.foundation.libraries.batch.core.base.messageTablePrefix=my_batch_
```

BatchEngineProperties

Configuration prefix: `nc.foundation.libraries.batch.engine.base`

Defines default configuration for the `batchEngineTaskExecutor` component, including thread pool sizes for job execution and partition processing.

Property	Default Value	Description
`jobWorkerThreads`	`4`	Number of threads for job execution and remote partitions

BatchSchedulerConsumerProperties

Configuration prefix: `nc.foundation.libraries.batch.scheduler.processor.base`

Property	Default Value	Description
`failedJobBaseRetryDelayMinutes`	`30`	Base delay in minutes for failed job retries (used in exponential backoff calculation)

Roles and Rights

> **Important:** The included SecurityRoles are intended for specific usage, please read carefully.

Role	Module location	Description
BATCH_READ	`batch-core-service`	Should be used for reading of data related to batch jobs and items.
BATCH_WRITE	`batch-core-service`	Should be used for writing of data related to batch jobs and items.
BATCH_SCHEDULER_READ	`batch-scheduler-service`	Should be used for reading of scheduler data.
BATCH_SCHEDULER_WRITE	`batch-scheduler-service`	Should be used for writing of scheduler data.

Database Patches

See configuration of system parameters for database patch required to specify timeout for a batch job.

FAQ

Why can I not find my batch job in the list of batch jobs?

When you have created a batch job and want to test it, you might not be able to find the batch job in the list of batch jobs in the administration UI. This is probably because the batch job is not registered in the application. To register the batch job, you need to create a variable with type `BatchJob`. To see an example, view a constant file of the batch job examples. This will initialize the batch job with the given name.

Each batch job is also a part of a batch configuration group; an example of that can be found in `BatchJobExamplesConfig`. Ensure you have a configuration where you import your

batch job.

How do I debug a batch job?

To debug a batch job, you can use the following steps:

1. Start the application in debug mode
2. Set a breakpoint in the batch job/batch item you want to debug
3. Run the batch job. See Getting started with Batch jobs.
4. The application will stop at the breakpoint, and you can debug the batch job

Why are my validators run twice in my batch jobs?

See the following GitHub issue on Spring Batch: https://github.com/spring-projects/spring-batch/issues/1452